MASTER APPLICATION BRIEF

Document Production Application

1. Project Overview

Build a professional, reliable, mobile-friendly document production application designed primarily for personal use.

The application must simplify the creation, processing, organization, editing and export of books and other documents.

It must have a polished, production-quality interface and reliable internal architecture. It must not feel like a temporary prototype or unfinished utility.

2. Homepage

The homepage must contain three distinct upload workflows:

1. Upload Document

- Accept one document at a time.
- Process the selected document before starting another document workflow.

2. Upload Images

- Accept individual images.
- Support batch image uploads.
- Provide image organization and processing capabilities.

3. Upload Spreadsheet

- Accept spreadsheet files.
- Automatically populate document pages using spreadsheet content.
- Allow subsequent editing and arrangement.

The three workflows must remain independent because they perform different functions.

3. Project Library

The homepage must also include a project library containing previously created or processed projects.

Users must be able to:

- Save projects.
- Automatically save changes.
- Reopen projects.
- Continue unfinished work.
- Replace uploaded content.
- Duplicate projects.
- Rename projects.
- Export projects.
- Delete projects and their associated stored files.

4. Data Persistence

The application must retain projects between sessions.

Initial recommended architecture:

- IndexedDB for local project storage.
- Store project metadata, source files, images, editing configurations and recovery information.
- Use transactional operations and safe file management.
- Support project backup and restoration.

A cloud database is not mandatory for the initial version.

Cloud synchronization may be considered as a future optional feature.

5. Reliability and Performance

The application must prioritize:

- Automatic saving.
- Recovery after unexpected interruptions.
- Efficient memory management.
- Responsive user interactions.
- Efficient handling of large documents and image batches.
- Reliable file importing and exporting.
- Clear error messages.
- Protection against accidental data loss.
- Consistent interface behaviour.

Heavy processing should not unnecessarily block the user interface. Use appropriate background processing and progress indicators.

6. Interface Quality

The application must have:

- A professional and consistent visual design.
- Clear navigation.
- Responsive layouts.
- Intuitive controls.
- Appropriate loading and success states.
- Useful empty states.
- Clear error and recovery states.

The user's approved Canva designs will serve as the visual reference for implementation.

7. Development Standards

- Build modularly.
- Avoid unnecessary dependencies.
- Do not introduce features that compromise existing functionality.
- Test individual workflows before integration.
- Preserve existing working features during updates.
- Test on mobile devices and desktop browsers.
- Never silently discard user data or substitute unsupported file formats.

8. Important Product Principle

This is primarily a personal production application, not a multi-user SaaS platform.

Do not introduce unnecessary user registration, subscriptions, payment systems or complicated backend infrastructure.

Prioritize functionality, reliability, data preservation and ease of use.

Application Storage and Backup Architecture

Storage architecture: Local-first.

Primary database: IndexedDB.

The application must not require a cloud database or user account to operate.

Automatic saving

- Automatically save project changes.
- Preserve uploaded source files and project configurations.

- Avoid losing unsaved work when the application closes unexpectedly.
- Use safe, transactional storage operations.

Automatic recovery and backups

- Maintain recoverable project snapshots.
- Restore projects after unexpected interruptions.
- Preserve the most recent successfully saved state.
- Handle storage errors without silently discarding user data.

Storage management

- Request persistent browser storage where supported.
- Monitor available storage capacity.
- Notify users when storage space becomes insufficient.
- Provide project backup export and restoration.

Future consideration

Support external backup destinations without making cloud storage mandatory.

The application must remain usable offline for its local production and project-management functions.

APPLICATION OVERVIEW AND WORKFLOW SUMMARY

The application has three content-generation workflows, followed by shared editing, export and storage functionality.

1. DOCUMENT WORKFLOW
DOCX → Spreadsheet → Templates → Auto-population → Editing → Export

1. Upload a DOCX document.
2. The system reads the document page by page and identifies content types such as titles, subtitles, headings, paragraphs and quotes.
3. It generates a spreadsheet containing the extracted content.
4. Download and review the spreadsheet.
5. Correct extraction errors and upload the revised spreadsheet.
6. Upload design templates.
7. The system matches spreadsheet rows to template pages and columns to their respective content types.
8. It automatically populates the templates.
9. Enter the editing interface to adjust populated pages.

10. Export the finished work.

Purpose: Convert existing documents into designed publications.

2. IMAGE WORKFLOW
Images → Automatic placeholders → Layout design → Apply to all → Spreadsheet → Auto-population → Editing → Export

1. Upload multiple images.
2. The application automatically places standard text placeholders, including title, subtitle and other content types.
3. Arrange the first page's layout.
4. Select Apply layout to all pages or to selected number of pages if required.
5. The application applies the same layout to all uploaded images.
6. Upload the spreadsheet.
7. The application matches spreadsheet columns to corresponding placeholders and rows to individual images.
8. It populates the images.
9. Enter the editing interface.
10. Adjust text positions, sizes, fonts, effects and other design elements.
11. Export the finished images.

Purpose: Produce large batches of graphics efficiently, including blog images and promotional designs.

3. SPREADSHEET WORKFLOW
Spreadsheet → Select template → Auto-population → Editing → Download

1. Upload a spreadsheet.
2. Select one of the application's existing templates.
3. The spreadsheet must follow the template's predefined column structure, such as Title, Subtitle and Subtext.
4. The application automatically populates the selected template.
5. Enter the editing interface.
6. Make necessary adjustments.
7. Download the finished document.

Purpose: Quickly generate documents from structured spreadsheet content.

4. SHARED APPLICATION FUNCTIONALITY
- Editing interface: Edit and arrange populated content.
- Text editing: Change text and content.
- Typography: Fonts, sizes and text styling.
- Text effects: Apply visual text effects.

- Layout editing: Reposition and organize elements.
- Export: Produce finished files.
- Storage: Keep and organize projects and outputs.

5. KEY WORKFLOW DISTINCTIONS
- Document workflow: Generates its spreadsheet from a DOCX.
- Image workflow: Creates placeholders and lets the user establish a reusable layout.
- Spreadsheet workflow: Uses predefined application templates.
- All three workflows have an editing stage after automatic population.
- The user controls final design adjustments.

6. DETAILED INTERFACE DESIGN
The next stage is to define the application screen by screen:
- Screen appearance.
- Controls and buttons.
- Button functionality.
- Actions and resulting behaviour.
- Navigation between screens.
- Features belonging to each workflow.
- Existing functionality to correct.
- New functionality to build.

Starting point: Main dashboard. Determine what users see when they first open the application.

Automatic Single and Bulk Upload Recognition

The upload system must accept one file or multiple files.

It must automatically detect how many files the user selected.

If one image is uploaded, process it as a single design.

If multiple images are uploaded, process them as a bulk design.

Apply the same principle to document and spreadsheet workflows where applicable.

Do not force users to choose Single or Bulk before uploading.

Do not restrict the upload input to either single-file or multiple-file selection.

The upload and processing system must work correctly in both cases, without errors or glitches.

LITDOM BATCH STUDIOS

Search here ...

Create a Project

Upload DOCX

Document

Upload Images

Images

Upload Sheet

Sheet

Your Projects



Your Design 1

Storage

Used:

250MB

Available:

1GB

Illustrative storage values and images for project previews not actual measurements

The app interface should look like the image above. That represents the landing page or general app dashboard.

LITDOM BATCH STUDIO

DOCUMENT ANALYSIS AND ADAPTIVE SPREADSHEET GENERATION SPECIFICATION

1. Purpose

The application must accept properly formatted Microsoft Word DOCX documents, identify their text styles, extract their contents, and automatically generate a structured spreadsheet.

The spreadsheet will organize the extracted content according to document pages and text styles.

The system must support documents originally created in either Microsoft Word or Google Docs, provided they are exported or saved in DOCX format with their formatting information preserved.

2. Required Upload Format

All documents submitted through the document-upload workflow must be in `.docx` format.

The application must read the DOCX document structure and formatting information to identify the styles assigned to individual paragraphs.

It must not rely solely on visual appearance, font size, boldness, or manually guessing the purpose of a paragraph.

The software used to create the original document must not determine whether the application can process it.

3. Base Sheet: Standard Style Categories

The application must maintain a standard Base Sheet containing the following 27 initial style categories:

1. Title
2. Subtitle

3. Heading 1
4. Heading 2
5. Heading 3
6. Heading 4
7. Heading 5
8. Heading 6
9. Heading 7
10. Heading 8
11. Heading 9
12. Normal Text
13. Quote
14. Intense Quote
15. Block Text
16. Body Text
17. Body Text 2
18. Body Text 3
19. Body Text First Indent
20. Body Text Indent
21. Caption
22. Book Title
23. List Paragraph
24. List Number
25. List Bullet
26. Note Heading
27. Plain Text

These are the initial supported style categories. The system should also be capable of identifying additional custom or built-in DOCX styles where possible.

4. Critical Requirement: Dynamic and Unlimited Style Duplication

Every style category must support multiple occurrences within a single document page.

There must be NO fixed maximum number of occurrences assigned to any style.

For example, a page may contain:

- Multiple subtitles.
- Multiple Heading 1 paragraphs.
- Multiple Heading 2 paragraphs.
- Multiple Heading 3 paragraphs.
- Multiple Heading 4 paragraphs.
- Multiple Heading 5 paragraphs.
- Multiple Heading 6 paragraphs.
- Multiple Heading 7 paragraphs.

- Multiple Heading 8 paragraphs.
- Multiple Heading 9 paragraphs.
- Multiple Normal Text paragraphs.
- Multiple quotes.
- Multiple captions.
- Multiple body text paragraphs.
- Multiple occurrences of any other supported style.

The same duplication principle applies to ALL 27 initial style categories.

5. Automatic Column Generation

The application must scan the entire uploaded DOCX document before generating the spreadsheet.

For each style category:

1. Identify every occurrence of that style on every page.
2. Count the occurrences separately for each page.
3. Determine the highest number of occurrences on any single page.
4. Automatically generate enough spreadsheet columns to accommodate that maximum.
5. Number the generated columns according to their occurrence order.

Examples:

- Subtitle 1, Subtitle 2, Subtitle 3…
- Heading 1 - 1, Heading 1 - 2, Heading 1 - 3…
- Heading 2 - 1, Heading 2 - 2…
- Normal Text 1, Normal Text 2, Normal Text 3…
- Quote 1, Quote 2, Quote 3…
- Caption 1, Caption 2, Caption 3…

This principle must work consistently for every supported style.

The number of generated columns must be determined by the actual document being processed, not by an arbitrary limit established in advance.

6. Spreadsheet Row Structure

Each document page must correspond to exactly one spreadsheet data row.

For example:

- A 10-page document produces 10 active data rows.
- A 100-page document produces 100 active data rows.
- A 300-page document produces 300 active data rows.

The first row contains the spreadsheet column headers.

Each subsequent row represents one corresponding page in the original document.

7. Automatic Content Population

After generating the required columns, the application must populate the spreadsheet.

Each extracted paragraph must be placed in the column corresponding to its assigned document style and its occurrence number on that page.

For example, if Page 1 contains three Normal Text paragraphs, they must populate:

- Normal Text 1
- Normal Text 2
- Normal Text 3

If Page 2 contains only one Normal Text paragraph, it must populate Normal Text 1 while leaving Normal Text 2 and Normal Text 3 empty.

The same rule applies to all other styles.

Unused cells must remain empty. The application must not shift content into unrelated columns to fill empty spaces.

8. Preserve Original Reading Order

The application must preserve the original sequence of content within each page.

Although the spreadsheet organizes content into style-based columns, the original reading order must also be retained through appropriate sequence information or metadata.

For example, if a page contains:

1. Title
2. Normal Text
3. Heading 1
4. Normal Text
5. Quote
6. Heading 2
7. Normal Text

The application must retain this exact sequence.

Column organization must never cause the original reading order to be lost.

9. Core Design Principle

The Base Sheet is a standard dictionary of document-style categories. It is NOT a fixed template that restricts how much content a document can contain.

The uploaded document determines the spreadsheet structure.

The application must dynamically expand the spreadsheet according to the actual styles and number of occurrences discovered during document analysis.

The document determines the required columns. The spreadsheet adapts to the document, not the other way around.

10. Reliability Requirements

- Never discard text because a style appears more times than expected.
- Never impose arbitrary occurrence limits.
- Never assume that every page has the same number of paragraphs or headings.
- Never confuse different heading levels.
- Never combine different styles into one column.
- Never overwrite repeated occurrences of the same style.
- Never populate unused cells with invented content.
- Preserve page boundaries and paragraph order.
- Support both Google Docs-originated and Microsoft Word-originated DOCX files.
- Preserve the distinction between paragraph styles and character formatting such as bold, italic, underline, and text colour.

11. Implementation Priority

This functionality must be implemented as a reliable document-analysis and spreadsheet-generation engine.

It must work with single-page documents and large multi-page documents.

Do not replace this adaptive system with a rigid spreadsheet containing a predetermined number of repeated style columns.

The system must be designed to accommodate the actual content of each uploaded document.

12. Variable Page Structures and Conditional Content Population

The application must understand that different pages within the same document can have completely different content structures.

Not every page is required to contain all supported document styles.

For example:

- A novel may contain only a title and normal text paragraphs.
- A textbook may contain titles, subtitles, multiple heading levels, paragraphs, quotations and numbered lists.
- An instructional document may contain headings, subtitles, numbered steps and bullet points.
- A page may contain several styles, while another page may contain only one or two.

The application must analyze each page independently.

Conditional Population Rules

1. Populate a spreadsheet cell only when the corresponding style and content actually exist on that page.
2. Leave cells empty when the corresponding style is absent.
3. Never invent content to fill an empty cell.
4. Never force a page to contain styles that do not exist in the original document.
5. Never generate placeholders for empty spreadsheet cells.
6. Preserve all existing content and its original page assignment.

Empty-Cell Interpretation

An empty cell means that the corresponding content is not present on that page.

For example, if a page contains only a Title and Normal Text, the application must populate only those corresponding cells and leave Subtitle, Heading 1, Heading 2, Quote, Caption and all other unused style cells empty.

When the application later populates design templates, it must create text elements only for cells containing actual content.

An empty cell must not produce an empty text box or placeholder.

Extraction Error Distinction

The application must distinguish between:

- A style that genuinely does not exist on a page.
- A style that exists but whose content could not be extracted successfully.

Extraction failures must be reported rather than silently interpreted as empty cells.

Core principle: Every page must retain its actual structure. The application must populate what exists, leave out what does not exist, and never impose a predetermined page layout on the document.

13. Consecutive Same-Style Content Grouping

The application must distinguish between individual sentences, paragraphs and groups of consecutive content sharing the same document style.

It must NOT automatically create a new spreadsheet occurrence for every sentence or paragraph.

Instead, it must group consecutive content elements that share the same document style into one occurrence.

Example 1: Novel

A page contains:

- Title: Chike Went to School
- Normal Text paragraph: Chike woke up early.
- Normal Text paragraph: He prepared for school.
- Normal Text paragraph: He packed his books.
- Normal Text paragraph: He left the house.

The application must combine the consecutive Normal Text paragraphs into one spreadsheet entry:

- Title
- Normal Text 1

Normal Text 1 contains all four paragraphs, preserving their original paragraph boundaries.

It must NOT create Normal Text 1, Normal Text 2, Normal Text 3 and Normal Text 4 simply because four paragraphs exist.

Example 2: Different styles interrupting one another

A page contains:

1. Normal Text
2. Normal Text
3. Heading 1
4. Normal Text
5. Normal Text
6. Quote
7. Normal Text

The application must produce:

- Normal Text 1: The first two consecutive Normal Text paragraphs.
- Heading 1 - 1: The heading.
- Normal Text 2: The next two consecutive Normal Text paragraphs.
- Quote 1: The quotation.
- Normal Text 3: The final Normal Text content.

Grouping Rules

1. Consecutive paragraphs with the same recognized style belong to one occurrence.
2. A change to another style ends the current group.
3. If the original style appears again after another style interrupts it, create a new numbered occurrence.
4. Preserve paragraph boundaries and the original reading order inside each group.
5. Do not merge content across page boundaries.
6. Apply this grouping principle to every supported document style, not only Normal Text.

Core principle: Count consecutive groups of the same style, not individual sentences or paragraphs. A new occurrence is created when a different style interrupts the existing group.

14. Global Shared Columns Across All Document Pages

The application must generate ONE shared spreadsheet column structure for the entire uploaded document.

It must NOT create a separate set of columns for each page.

Scanning and Column Generation

1. Scan every page of the uploaded DOCX.
2. Identify and group consecutive content blocks sharing the same document style.
3. Count the resulting style occurrences separately on each page.
4. Determine the maximum occurrence count for each style across all pages.
5. Generate one shared set of columns for each style, using that maximum count.
6. Reuse those exact columns for every page in the document.

Example

Suppose the scan discovers:

Style	Page 1	Page 2	Page 3	Maximum

Heading 1	3	2	1	3
Heading 2	2	4	1	4
Normal Text	3	2	5	5
Quote	5	3	2	5

The application generates:

- Heading 1 - 1, Heading 1 - 2, Heading 1 - 3.
- Heading 2 - 1, Heading 2 - 2, Heading 2 - 3, Heading 2 - 4.
- Normal Text 1 through Normal Text 5.
- Quote 1 through Quote 5.

These columns are created once and shared by all three page rows.

Page Population

Each page fills the shared columns according to its own detected content.

If Page 2 contains only two Heading 1 groups, it fills Heading 1 - 1 and Heading 1 - 2, leaving Heading 1 - 3 empty.

The application must never generate duplicate sets of column headers for different pages.

Core Principle

The entire document determines the spreadsheet's column structure. Individual pages determine what gets entered into each row.

A single shared column structure serves every page, regardless of whether their content structures are identical or different.

DOCUMENT WORKFLOW: ADAPTIVE SPREADSHEET GENERATION

1. Core Requirement

The application must generate a spreadsheet dynamically based on the actual content and formatting structures detected inside the uploaded DOCX document.

The spreadsheet must reflect the document's real structure. It must not automatically include every style category in the application's Base Sheet.

The Base Sheet contains the 27 initial supported style categories. These categories serve as a recognition dictionary for document analysis. They do not represent mandatory spreadsheet columns.

The uploaded document determines the spreadsheet structure.

2. Scan the Entire Document Before Generating the Spreadsheet

After the user uploads a DOCX document and selects "Generate Sheet", the application must:

1. Read and analyze the entire document.
2. Identify all document pages.
3. Detect the paragraph styles used throughout the document.
4. Identify the content belonging to each style.
5. Group consecutive content blocks with the same style into individual occurrences.
6. Count the occurrences of each style on every page.
7. Determine which styles are actually present in the document.
8. Calculate the maximum number of occurrences of each detected style across all pages.
9. Generate one shared spreadsheet structure using only the necessary style columns.

The application must complete this analysis before generating the downloadable spreadsheet.

3. Include Only Styles Detected in the Document

The application must not include unused style categories in the generated spreadsheet.

For example, suppose a 100-page document contains only these four styles:

- Title
- Subtitle
- Heading 1
- Heading 2

The generated spreadsheet must contain only these four style categories, with the necessary numbered occurrence columns.

It must not include unused categories such as:

- Quote
- Intense Quote
- Heading 9
- Plain Text
- Note Heading
- Caption
- List Paragraph
- Any other style that does not occur in the document.

This rule applies to all 27 initial supported categories.

If the document contains only two recognized styles, the spreadsheet should contain only those two styles. If it contains fifteen, it should contain only the fifteen detected styles.

Do not add unnecessary columns merely because those categories exist in the Base Sheet.

4. Generate Only the Required Number of Occurrence Columns

For each detected style, calculate the maximum number of consecutive same-style groups occurring on any single page.

Generate enough numbered columns to accommodate that maximum.

For example:

- Page 1 contains 3 Heading 1 groups.
- Page 2 contains 2 Heading 1 groups.
- Page 3 contains 1 Heading 1 group.

The spreadsheet needs:

- Heading 1 - 1
- Heading 1 - 2
- Heading 1 - 3

It does not need additional Heading 1 columns.

Apply this calculation independently to every detected style.

There must be no arbitrary fixed occurrence limit.

5. Use One Shared Column Structure Across the Entire Document

Generate the spreadsheet columns once, using the analysis of the entire document.

Do not generate a different set of columns for individual pages.

Every page must use the same shared spreadsheet columns.

For example, if the document requires:

- Title - 1
- Subtitle - 1
- Heading 1 - 1 through Heading 1 - 3
- Heading 2 - 1 through Heading 2 - 4
- Normal Text - 1 through Normal Text - 5

Those columns must remain the same for every page.

A page with fewer occurrences simply leaves the unused cells empty.

Do not regenerate or rearrange the columns for individual pages.

6. Rows Must Represent Document Pages

The spreadsheet must use this structure:

- Row 1: Column headers.
- Row 2: Content from document page 1.
- Row 3: Content from document page 2.
- Row 4: Content from document page 3.
- Continue until every document page has been represented.

For a document containing 100 pages:

- There must be 100 data rows.
- The first data row is row 2.
- The final data row is row 101.

The spreadsheet must not generate unnecessary additional data rows.

Each document page must have exactly one corresponding spreadsheet row.

7. Populate Each Page Using Its Own Detected Content

Each row must contain the content extracted from its corresponding document page.

For example, suppose the generated spreadsheet has these four columns:

Title	Subtitle	Heading 1	Heading 2

The content might look like this:

Title	Subtitle	Heading 1	Heading 2
Page 1 title			
	Page 2 subtitle	Page 2 heading	Page 2 subheading
Page 3 title		Page 3 heading	

The table above is illustrative. The actual spreadsheet must use the numbered occurrence columns required by the document.

The application must populate only the cells corresponding to content actually detected on each page.

8. Empty Cells Must Remain Empty

An empty cell means that the corresponding style occurrence does not exist on that particular page.

For example:

- If page 1 has a Title but no Subtitle, the Subtitle cell remains empty.
- If page 2 has a Heading 1 but no Heading 2, the Heading 2 cell remains empty.
- If page 3 has no Quote, its Quote cells remain empty.

Do not insert placeholder text, default headings, invented content, or unnecessary formatting into empty cells.

During subsequent template population, the application must not create text elements for empty spreadsheet cells.

9. Distinguish Missing Content From Extraction Failure

The application must not confuse an absent style with a failed extraction.

There are three different situations:

1. A style genuinely does not occur in the document.
2. A style exists in the document but does not occur on a particular page.
3. Content exists but the application fails to extract or classify it correctly.

The first two situations should produce the appropriate absence of content and empty cells.

The third situation must be identified and reported for review. It must not be silently treated as an empty cell.

10. Preserve the Original Content and Reading Order

While organizing content into style-based columns, the application must preserve:

- Original wording.
- Original paragraph boundaries.
- Original reading order.
- Original page boundaries.
- The relationship between each content group and its detected style.

The spreadsheet organization must not cause content to be lost, merged incorrectly, duplicated, or reordered.

The application must retain enough information to reconstruct each page’s original content sequence during later template population and editing.

11. Accuracy and Document Fidelity

Accuracy is a critical requirement.

The application must analyze the actual DOCX document structure and its available style metadata rather than relying only on visual appearance, font size, boldness, or other superficial formatting.

It must correctly distinguish between:

- Different style categories.
- Multiple occurrences of the same style.
- Consecutive paragraphs belonging to one occurrence.
- A new occurrence of a style appearing after another style interrupts it.
- Content belonging to different pages.

It must not silently discard text or classify content arbitrarily.

12. Main Operating Principle

The spreadsheet must be generated according to the actual document, not according to a fixed spreadsheet template.

Remember these three rules:

Rows represent pages.

Columns represent detected document styles and their required occurrences.

The uploaded document determines which columns are necessary and how many are needed.

The objective is to give the user a clean, accurate, document-specific spreadsheet containing all the extracted content without unnecessary columns, unnecessary rows, or unnecessary work during manual review.

LITDOM BATCH STUDIO: PRODUCT VISION AND CORE PURPOSE

1. Product Vision

Litdom Batch Studio is designed to make high-quality digital publishing and bulk content production faster, more accessible, and more efficient.

The application should enable users to create professionally designed digital products within minutes rather than spending hours or days manually designing and arranging individual pages.

For example, a user may want to create a high-quality book containing their knowledge, experiences, research, ideas, or original writing.

Instead of manually recreating every page in a design application, the user should be able to upload their document, allow the application to analyze and organize its content, select suitable design templates, automatically populate those templates, make any necessary adjustments, and export the finished publication.

The objective is to make a process that might ordinarily take two days achievable in approximately 30 minutes, depending on the document's complexity and the user's requirements.

2. Speed Without Sacrificing Quality

The application must not treat speed as permission to produce poorly designed or repetitive results.

Its purpose is to reduce repetitive manual work while maintaining the quality of the finished product.

Users should be able to produce:

- Well-designed books.
- Digital publications.
- Social media graphics.
- Pinterest pins.
- Educational materials.
- Other template-based digital products.

The finished products should be properly arranged, readable, visually appealing, and suitable for their intended purpose.

3. Bulk Production and Creative Variety

The application must support the production of large quantities of content without forcing every output to look identical.

For example, a user producing hundreds of Pinterest pins should be able to use different designs, layouts, backgrounds, typography, and visual arrangements.

The outputs should not look like repetitive copies of one another simply because they were generated in batches.

Bulk production must therefore combine:

- Automation.
- Design flexibility.
- Template variety.
- User editing control.
- Consistent output quality.

The objective is to let users create as many products as their workflow requires, without having to design every individual item from scratch.

4. Accessibility

The application is intended to make efficient digital production accessible to people who cannot afford expensive publishing or design software.

Users should not need expensive subscriptions or advanced design expertise to produce quality work.

The application should reduce unnecessary costs, technical barriers, and repetitive manual tasks while providing a practical production workflow.

5. Core Product Principle

Create more in less time, without sacrificing quality, originality, or creative control.

Litdom Batch Studio should make it possible for a user to spend more time developing valuable content and less time performing repetitive production work.

Automation should handle the repetitive operations. The user should retain control over the creative decisions and final result.

DOCUMENT WORKFLOW: REVISED SPREADSHEET PROCESSING, AUTOMATIC PLACEHOLDER PREPARATION, APPROVAL, AND CONTENT POPULATION

1. Purpose of This Stage

After the user has uploaded the original DOCX, received the generated spreadsheet, reviewed and revised it, and uploaded the design templates, the application proceeds to the final preparation stage.

The purpose of this stage is to prepare the templates for content population before inserting the actual spreadsheet content.

The application must first understand:

- How many pages the project requires.
- How many template images have been uploaded.
- Which content categories exist in the revised spreadsheet.
- How many occurrences of each category are required.
- Which spreadsheet columns belong to each page.
- Which placeholders need to be created on each template.

The application then prepares the placeholders automatically.

2. Process the Revised Spreadsheet

When the user uploads the revised spreadsheet, the application must compare it with the original project's background record.

It must recognize the spreadsheet as the revised version of the spreadsheet originally generated for that document project.

The application must examine the revised spreadsheet and identify:

- Current column headers.
- Added columns.
- Removed columns.
- Renamed columns.
- Changes to existing content.
- Number of data rows.
- Content distribution across rows.
- Number of pages represented.
- Content volume in each row and column.

The revised spreadsheet is the authoritative source for the content structure and corrections that must be reflected in the final template preparation.

3. Automatically Update the Placeholder Structure

The application must compare the revised spreadsheet with the original recorded structure.

There are two possible situations:

Situation A: No structural changes

If the user has only corrected or improved the content without changing the spreadsheet structure, the application should retain the previously determined placeholder structure.

It must not unnecessarily rebuild or duplicate the existing placeholders.

It should proceed using the existing structure.

Situation B: Structural changes

If the user has added, removed, renamed, or reorganized columns, the application must update the placeholder structure accordingly.

For example:

- If a new Heading 2 column has been added, prepare the corresponding additional placeholder.
- If an unnecessary column has been removed, remove its corresponding placeholder requirement.
- If a column has been renamed, update the relevant matching information.
- If two previously separate content categories have been corrected, adjust their corresponding placeholder assignments.

The application must not blindly use the original column structure when the revised spreadsheet has legitimately changed.

It must adapt to the revised spreadsheet.

4. Automatically Prepare Placeholders on the Templates

After processing the revised spreadsheet, the application must prepare the required placeholders on the uploaded template images.

The spreadsheet determines the content categories and the required number of placeholders.

The application must use:

- Spreadsheet columns to determine the types of placeholders required.
- Spreadsheet rows to determine which page each placeholder belongs to.
- Template images to determine where and how the placeholders can be arranged.

For example, if the revised spreadsheet contains Title, Subtitle, Heading 1, and Normal Text columns, the application must prepare the corresponding text fields on the appropriate templates.

If page 1 contains only a Title and Normal Text, its template should receive only the corresponding placeholders.

If page 2 contains a Title, Subtitle, and three Heading 1 occurrences, its template should receive the corresponding placeholders for those occurrences.

Do not create unnecessary placeholders for empty spreadsheet cells.

5. Match Placeholders to the Correct Pages

The application must maintain the page-to-row relationship.

For a 100-page document:

- There are 100 spreadsheet data rows.
- There must be 100 corresponding template images.
- Each spreadsheet row must control the placeholders prepared on its corresponding template.

The application must not mix content categories or placeholders between pages.

The same column structure may be shared across the entire spreadsheet, but each page receives only the placeholders required by its own row.

6. Use Content Volume to Prepare the Placeholders Intelligently

Placeholder preparation must account for the amount and type of content expected in each text field.

The application should consider:

- Word count.
- Character count.
- Paragraph count.
- Content hierarchy.
- Available template space.
- Text-field dimensions.
- Expected text overflow.

It should use these details to prepare appropriate text-field dimensions, positioning, and initial text sizing.

The purpose is to avoid blindly placing identical text boxes on every page regardless of the actual content.

The placeholders should be prepared with the expected content in mind.

7. Placeholder Review Stage

After preparing the placeholders, the application must display the templates for the user to review.

At this point, the actual spreadsheet content has not yet been populated.

The user should be able to inspect:

- Whether the correct placeholders were created.
- Whether each placeholder is on the correct page.
- Whether the correct content categories were assigned.
- Whether the text fields are positioned properly.
- Whether the placeholder dimensions are appropriate.
- Whether the initial layout is visually satisfactory.

The application must not automatically proceed to full content population without the user's approval.

8. Approval Before Content Population

Provide an explicit **Approve** or **Okay** action.

When the user approves the prepared placeholders, the application must proceed to populate the templates using the revised spreadsheet.

Approval is the transition between placeholder preparation and actual content population.

Do not treat uploading the spreadsheet as automatic approval.

The user must be given an opportunity to inspect the prepared pages first.

9. Editing Before and After Approval

The user must be able to make design adjustments both before and after approving the placeholders.

Before approval

The user can inspect and adjust the prepared placeholder layout before allowing the application to populate the content.

After approval

Once the content has been populated, the user can continue editing the finished pages.

The editing interface should provide the relevant design controls, including:

- Fonts and typography.
- Font sizes.
- Font colors.
- Text styles.
- Bold, italic, and other formatting.
- Text positioning.
- Text-field dimensions.
- Alignment.
- Spacing.
- Other available design adjustments.

The user must retain control over the appearance of the finished publication.

10. Actual Content Population

Once the user selects Approve, the application must populate the prepared placeholders using the revised spreadsheet.

It must:

- Match each spreadsheet row to its corresponding page.
- Match each spreadsheet column to its appropriate placeholder.
- Insert the correct content into the correct text field.
- Preserve the content's paragraph structure and reading order.
- Apply suitable initial typography and sizing.
- Respect the prepared placeholder dimensions.
- Handle empty cells without inserting invented content.

The application must use the revised spreadsheet, not the original extracted content, for population.

11. Final Editing and Export

After population, the user can make final adjustments to the text and design.

They can correct punctuation, edit wording, change fonts, adjust colors, resize text, reorganize layouts, and make any other necessary improvements.

When satisfied, the user exports the completed publication.

12. Required Workflow Sequence

The application must follow this sequence:

1. Generate the initial spreadsheet.
2. Download the spreadsheet.
3. Record the original spreadsheet structure in the project background.
4. Upload and retain the design templates.
5. User reviews and revises the downloaded spreadsheet.
6. User uploads the revised spreadsheet.
7. Application validates and processes the revised spreadsheet.
8. Application identifies structural changes.
9. Application updates the placeholder requirements.
10. Application prepares placeholders on the corresponding templates.
11. User reviews the prepared placeholders.
12. User may edit the placeholder layout.
13. User selects Approve.
14. Application populates the placeholders with the revised spreadsheet content.
15. User performs final editing.
16. User exports the completed publication.

13. Critical Operating Principle

Prepare first. Review second. Approve third. Populate fourth. Edit and export last.

The application must not confuse placeholder preparation with content population.

The purpose of automatic placeholder preparation is to ensure that the correct text fields exist on the correct pages before the actual content is inserted.

The user must retain the ability to inspect and adjust the prepared layout before approving population.

Litdom Batch Studio

Complete app workflow and document workflow specification

1. Purpose of the application

Litdom Batch Studio is a tool for creating and producing high-quality digital publications and visual content in batches.

Its purpose is to make digital publishing and content production faster, more accessible, and less dependent on expensive software subscriptions.

For example, a creator should be able to turn their knowledge, experience, research, and writing into a well-designed digital publication in a fraction of the time it would normally take to prepare everything manually. A task that might take two days could potentially be completed in approximately 30 minutes, depending on its complexity.

The application should support:

- Books and other digital publications.
- Pinterest pins and social media graphics.
- Educational materials.
- Information-based graphics.
- Other products that can be created using templates and structured content.

Core principle

Create more in less time, without sacrificing quality, originality, or creative control.

The application must not sacrifice content quality for speed. It should give creators control over their designs and allow them to produce varied, original work rather than making every output look like an identical copy.

Creators should be able to use different templates, layouts, backgrounds, typography, and other design elements.

The application is a production tool, not a replacement for the creator's knowledge or judgement. Content should be enriched with research, personal experience, practical insights, technical details, explanations, examples, updates, and editorial decisions where appropriate.

2. Existing application and implementation approach

The existing application is Litdom Batch Studio:

https://southecoast-creator.github.io/Litdom-Bulk-App-Create/

It was built using Google AI Studio and GitHub.

The application already has an image workflow. The document and spreadsheet workflows should be integrated into the existing application.

Do not replace the application with an entirely new application.

Before making changes:

- Inspect the existing code and application structure.
- Understand how the current image workflow works.
- Preserve existing working features.
- Integrate the new functionality into the existing architecture.
- Avoid unnecessary redesigns or changes to unrelated parts of the application.
- Avoid introducing paid infrastructure or unnecessary services.
- Preserve compatibility with the existing deployment.

The existing visual design is important. The dashboard and popup designs have been prepared separately. Do not redesign them or substitute a different interface without being instructed to do so.

3. The three main workflows

The application has three main workflows.

Workflow 1: Document workflow

DOCX → Spreadsheet generation → Template upload → Spreadsheet revision → Revised spreadsheet upload → Validation → Placeholder preparation → Placeholder review and editing → Approval → Content population → Final editing → Export.

This workflow converts a structured document into an editable spreadsheet, then uses the revised spreadsheet to populate corresponding design templates.

The detailed requirements are provided in Section 5.

Workflow 2: Image workflow

Images → Automatic placeholders → Layout design → Apply layout to all pages → Spreadsheet upload → Automatic population → Editing → Export.

This is an existing workflow and must be preserved.

The process is:

1. The user uploads one or multiple images.
2. The application prepares standard text placeholders automatically.
3. The user arranges the placeholders on the first image.
4. The user selects **Apply layout to all pages**.
5. The application applies that layout to the other uploaded images.
6. The user uploads a spreadsheet.
7. Spreadsheet columns are matched to the relevant placeholders.
8. Spreadsheet rows provide the content for the corresponding images.
9. The application populates the images.
10. The user edits the resulting designs and exports them.

The user should be able to edit text, positions, dimensions, fonts, font sizes, effects, and other supported design properties.

Do not replace this existing workflow with the document workflow. They are separate workflows.

Workflow 3: Spreadsheet workflow

Spreadsheet → Select an existing application template → Automatic population → Editing → Download.

The user uploads a spreadsheet that follows the selected template's predefined column structure.

The application uses the spreadsheet content to populate the selected template.

The user can edit the resulting designs and download them.

This workflow is separate from the document workflow, where the application generates the spreadsheet from an uploaded DOCX.

4. Dashboard and project management

The dashboard has three main upload options:

1. Upload Document
2. Upload Images
3. Upload Spreadsheet

There is also a **Your Projects** section.

Upload behaviour

The application should accept one or multiple files without requiring the user to choose between Single Design and Bulk Design first.

The number of uploaded files should determine how the application processes them.

For example:

- One image should work.
- Multiple images should work.
- One DOCX should work, whether it contains one page or many pages.
- One spreadsheet should work, whether it contains one data row or many data rows.

Do not make the application expect multiple files when the user uploads one. Likewise, do not make it expect only one file when the user uploads multiple files.

The earlier idea of forcing users to select Single Design or Bulk Design through a preliminary popup was removed.

Your Projects

The dashboard should display saved projects.

Each project should have a three-dot menu containing:

- Download
- Edit
- Delete

The storage display should show used and remaining storage.

When a project is deleted:

- Its associated project data should be deleted.
- Its stored project files should be removed.
- The available storage should update accordingly.

Deleting a project should free its associated storage.

5. Detailed document workflow

This is the current focus of development.

Stage 1: Upload a DOCX document

The document workflow accepts DOCX files.

The user may create the document in Google Docs or Microsoft Word, provided the document is exported as DOCX with its formatting and style information preserved.

The application must inspect the actual DOCX structure.

It should not depend only on visual appearance, font size, boldness, or other formatting clues to determine the document's structure.

The purpose of this stage is to understand:

- How many pages the document contains.
- Which recognized styles are used.
- Where each style occurs.
- How many times each style occurs on each page.
- Which content belongs together.
- The order of the content.
- Which page each content block belongs to.

The application must inspect the entire document before generating the spreadsheet.

Stage 2: Base Sheet and the 27 style categories

The application has one Base Sheet containing 27 initial style categories.

These categories form a style-recognition dictionary.

They are not a fixed spreadsheet structure, and they are not instructions to create all 27 columns automatically.

The 27 initial categories are:

1. Title
2. Subtitle
3. Heading 1
4. Heading 2
5. Heading 3
6. Heading 4
7. Heading 5
8. Heading 6
9. Heading 7
10. Heading 8
11. Heading 9
12. Normal Text
13. Quote
14. Intense Quote
15. Block Text
16. Body Text
17. Body Text 2

18. Body Text 3
19. Body Text First Indent
20. Body Text Indent
21. Caption
22. Book Title
23. List Paragraph
24. List Number
25. List Bullet
26. Note Heading
27. Plain Text

These are the initial supported categories.

The application should also be capable of identifying additional built-in or custom styles where possible. Unknown styles must not automatically be forced into an unrelated category.

Character formatting such as bold, italic, underline, and text colour is separate from paragraph styles.

Stage 3: Scan the entire document

The application must scan the entire DOCX before generating the spreadsheet.

It must identify which of the 27 recognized styles actually appear in the document.

Only styles that are genuinely used should become spreadsheet columns.

For example, if a document contains:

- Title
- Subtitle
- Heading 1
- Heading 2
- Normal Text

Then the generated spreadsheet should contain columns for those styles only, with additional numbered columns where repeated occurrences require them.

It must not automatically include unused styles such as Quote, Heading 9, Note Heading, or Plain Text.

The Base Sheet is a recognition dictionary, not a requirement to include every category in every spreadsheet.

Stage 4: Determine the required number of columns

The application must calculate how many occurrences of each style appear on each page.

It must then determine the maximum number of occurrences of each style on any single page.

That maximum determines how many shared columns are needed for that style across the entire document.

For example:

Style	Page 1	Page 2	Page 3	Required columns
Heading 1	3	2	1	3
Heading 2	2	4	1	4
Normal Text	3	2	5	5
Quote	5	3	2	5

The resulting columns would be:

- Heading 1 - 1
- Heading 1 - 2
- Heading 1 - 3
- Heading 2 - 1
- Heading 2 - 2
- Heading 2 - 3
- Heading 2 - 4
- Normal Text - 1
- Normal Text - 2
- Normal Text - 3
- Normal Text - 4
- Normal Text - 5
- Quote - 1
- Quote - 2
- Quote - 3

- Quote - 4
- Quote - 5

The numbering identifies the occurrence of a style. It does not introduce a new style category.

Important rules:

- Columns are generated dynamically.
- The same shared columns are used for every page.
- Do not generate a separate column structure for each page.
- Do not impose an arbitrary maximum number of occurrences.
- Do not restrict the user to a fixed number of columns.
- Generate only the columns required by the document.
- Do not create unnecessary columns for styles that do not appear.

Stage 5: Group consecutive content using the same style

The application must not treat every paragraph or sentence as a separate occurrence.

Consecutive content blocks using the same recognized style must be grouped into one occurrence.

A different style interrupts the group.

When the original style appears again after an interruption, it becomes the next numbered occurrence.

Example:

Content order	Recognized style
1	Normal Text
2	Normal Text
3	Heading 1

4	Normal Text
5	Normal Text
6	Quote
7	Normal Text

Expected grouping:

- Normal Text - 1: the first two consecutive Normal Text blocks.
- Heading 1 - 1: the Heading 1 content.
- Normal Text - 2: the next two consecutive Normal Text blocks.
- Quote - 1: the Quote content.
- Normal Text - 3: the final Normal Text block.

This rule applies to all recognized styles.

Preserve the original paragraph boundaries and reading order within each group.

Do not merge content across page boundaries.

The word “paragraph” describes a content unit in the document. It must not be added to the style name. The style name remains **Normal Text**.

Stage 6: Assign document pages to spreadsheet rows

Each document page corresponds to exactly one spreadsheet data row.

Row 1 contains the column headers.

Row 2 represents page 1.

Row 3 represents page 2.

Row 4 represents page 3.

This continues until the final document page.

For example, a 100-page document must produce:

- One header row.
- 100 data rows.
- Page 1 in row 2.
- Page 100 in row 101.

The spreadsheet must not contain unnecessary extra data rows.

The application must preserve page boundaries and correctly assign content to its corresponding page.

It must not assume that one paragraph equals one page.

Page breaks and section breaks must be considered. Where page assignment depends on rendered pagination, the application must use a reliable pagination or rendering method and validate the result.

If the actual page boundaries cannot be established reliably, the application must flag the problem instead of silently assigning content to the wrong page.

Stage 7: Populate the generated spreadsheet

Once the application has completed style detection, occurrence grouping, page assignment, and column calculation, it generates an actual Excel spreadsheet.

The spreadsheet must contain:

- One header row.
- The dynamically generated style columns.
- One data row for each document page.
- The corresponding content in the appropriate cells.

Each cell represents the content associated with a particular style occurrence on that page.

For example, if page 1 contains only Title and Normal Text, only the corresponding cells should contain content.

All other cells for that page remain empty.

If page 2 contains Title, Subtitle, and three Heading 1 occurrences, those corresponding cells should be populated.

Stage 8: Empty cells and missing content

An empty cell means the corresponding content genuinely does not exist on that page.

It does not mean that the application failed to extract the content.

Rules:

- If a style does not appear on a page, its corresponding cells remain empty.
- If a style appears fewer times than the total number of shared columns, the unused occurrence cells remain empty.
- Do not insert placeholder text into empty cells.
- Do not invent content.
- Do not copy content from another page to fill an empty cell.
- Do not create text elements for empty cells during template population.

If the application fails to extract content that should exist, it must identify and report the extraction failure. It must not silently represent an extraction failure as a genuinely empty cell.

Stage 9: Generate and download a real Excel file

The application must generate a real, usable Excel spreadsheet.

It is not sufficient to show a spreadsheet preview or display a download-success message without actually saving a file.

The user must be able to download the file to their device, including a mobile device.

The downloaded file must be usable in Google Sheets or another compatible spreadsheet application.

The user must be able to:

- Open the spreadsheet.
- Review the content.
- Edit existing cells.
- Correct extraction mistakes.
- Add missing content.
- Expand explanations.
- Improve instructions.
- Add research and technical details.
- Add personal experience and practical examples.
- Update information.
- Make other legitimate improvements.

The generated spreadsheet is an editable working copy, not the finished publication.

Stage 10: Record the original spreadsheet structure in the project

After generating the initial spreadsheet, the application should retain a background record of the original spreadsheet structure.

This record belongs to the current project.

It should preserve the original column headers and relevant project information so the application can understand the original spreadsheet structure when the revised spreadsheet is uploaded later.

The background record must remain associated with the project.

It should only be deleted when the user deletes the project.

The record helps the application identify the original document structure and understand the relationship between the original spreadsheet, its pages, and the uploaded templates.

However, the original record must not override the revised spreadsheet.

The revised spreadsheet is authoritative for the actual content and legitimate structural corrections.

Stage 11: Upload templates

After the initial spreadsheet has been generated and downloaded, the interface should show an **Upload Templates** section beneath it.

The user uploads template images one by one.

These images are the designs that will receive the document content.

The templates must be stored within the current project.

The upload order must be preserved because the templates correspond to the document pages.

The number of uploaded templates must match:

- The number of original document pages.
- The number of spreadsheet data rows.

For example, a 100-page document requires 100 template images and 100 spreadsheet data rows.

The application must validate the count.

If the counts do not match, it must report the mismatch instead of silently skipping, duplicating, or inventing pages.

Important:

At this stage, the application must not insert placeholders or perform final column-to-text-field matching.

The user has not yet revised the spreadsheet.

The initial spreadsheet may contain extraction errors or content that needs correction. The user must be allowed to review and revise it before the application prepares the final placeholders.

Templates should be retained in the project while the spreadsheet is being reviewed.

Stage 12: Revise the generated spreadsheet

The user opens the downloaded Excel file in Google Sheets or another compatible spreadsheet application.

The user reviews and improves the spreadsheet.

The user should be able to:

- Correct incorrectly extracted content.
- Correct content placed under the wrong style.
- Add missing content.
- Expand explanations.
- Improve paragraphs.
- Add examples.
- Add research.
- Add technical information.
- Add personal experience.
- Update information.
- Add columns when necessary.
- Remove columns when necessary.
- Rename or reorganize columns when necessary.
- Make other legitimate structural changes.

The application must not assume that the original spreadsheet is perfect.

The user’s revised spreadsheet is the source of truth for the next stage.

Stage 13: Upload the revised spreadsheet

The user uploads the revised version of the spreadsheet into the same project.

It is the same generated spreadsheet that the user downloaded and edited, not an unrelated spreadsheet.

The user should retain the same filename when uploading the revised file.

The application must associate the revised spreadsheet with the correct project.

The filename is a useful identifier, but it must not be the only way the application verifies the relationship. The project association and spreadsheet structure must also be checked.

The application should compare the revised spreadsheet with the background record of the original spreadsheet.

It must identify:

- Columns that remain unchanged.
- Columns that have been added.
- Columns that have been removed.
- Columns that have been renamed.
- Columns that have been reorganized.
- Changes in the number of data rows.
- Other relevant structural changes.

Legitimate changes must not be automatically rejected.

If the application cannot determine how a changed column relates to the original structure, it must flag the ambiguity instead of silently guessing.

The revised spreadsheet remains authoritative for the actual content.

Stage 14: Validate and process the revised spreadsheet

Uploading the revised spreadsheet does not mean that the application should immediately populate the templates.

The application must first validate and process the revised spreadsheet.

It must:

- Verify the spreadsheet's association with the current project.
- Check the column structure.
- Identify structural changes.
- Check the data rows.
- Validate the relationship between rows, pages, and templates.
- Identify empty cells.
- Identify ambiguous or unrecognized columns.
- Determine the updated placeholder requirements.

The application must not silently discard user edits or overwrite the revised spreadsheet with the original extraction.

If the revised spreadsheet contains a structural ambiguity that prevents reliable matching, the application must report it and request correction where necessary.

Stage 15: Prepare placeholders

Placeholder preparation happens only after the revised spreadsheet has been uploaded and validated.

The application uses:

- The revised spreadsheet's columns to determine the required content fields.
- The revised spreadsheet's rows to determine which page receives the content.
- The uploaded template images to determine where and how the fields can fit.

The application must prepare placeholders on the corresponding templates.

It must not insert the spreadsheet's actual content at this stage.

It must prepare the structure that will receive the content after the user approves it.

For example:

If page 1 has content only under Title and Normal Text, the corresponding template should receive only the required Title and Normal Text placeholders.

If page 2 has Title, Subtitle, and three Heading 1 occurrences, its template should receive the corresponding placeholders.

Empty spreadsheet cells must not generate placeholders.

The application must match each spreadsheet row to its corresponding template image using the established page order.

Stage 16: Update placeholders when the spreadsheet structure changes

The application must compare the revised spreadsheet with the original project record.

If the spreadsheet structure has not changed, the application should retain the previously determined placeholder structure rather than unnecessarily rebuilding or duplicating it.

If the user has added, removed, renamed, or reorganized columns, the application must update the placeholder requirements accordingly.

The application must not create duplicate placeholders because it has processed the same structure more than once.

It must not continue using outdated placeholder requirements when the revised spreadsheet has legitimately changed.

The revised spreadsheet determines the required content structure.

Stage 17: Account for content size and available template space

Placeholder preparation must consider the actual content requirements.

The application should evaluate:

- Word count.
- Character count.
- Number of paragraphs.
- Content hierarchy.
- The available dimensions of the relevant template area.
- Expected text overflow.
- Appropriate text sizing and positioning.

The application should prepare placeholders that are suitable for the content they will receive.

It must not automatically shrink text to unreadable sizes just to force content into a template.

If content is likely to exceed the available space, the application must flag the expected overflow.

The user must be able to adjust the placeholder layout before approval.

Stage 18: Review and edit prepared placeholders

After preparing the placeholders, the application presents the templates for user review.

The user must be able to inspect the prepared placeholders before any spreadsheet content is inserted.

The user can edit the placeholder layout, including:

- Position.
- Width and height.
- Alignment.
- Dimensions.
- Spacing.
- Font.

- Font size.
- Text styling.
- Other supported layout properties.

The user must be able to make corrections to the prepared layout before proceeding.

Stage 19: Approval

The user must explicitly approve the prepared placeholders.

There should be an approval action such as **Approve** or **Okay**.

Uploading the revised spreadsheet is not approval.

Preparing the placeholders is not approval.

Reviewing the placeholders is not approval.

The application must wait for the user's approval before inserting the actual spreadsheet content into the templates.

Stage 20: Populate the templates

Only after the user approves the prepared placeholders should the application populate them with the revised spreadsheet content.

The application must:

- Use the revised spreadsheet, not the original extraction.
- Match each spreadsheet row to the corresponding template page.
- Match each column to its corresponding placeholder.
- Preserve the content's reading order.
- Preserve paragraph boundaries.
- Keep content in its correct style field.
- Leave genuinely empty cells empty.
- Avoid creating content for empty cells.
- Avoid duplicating or skipping content.

The populated templates should correspond to the revised spreadsheet and the correct document pages.

Stage 21: Edit populated pages

After population, the user should be able to edit the resulting pages.

Editing controls should support:

- Typing and replacing text.
- Changing fonts.
- Changing font sizes.
- Changing text colours.
- Applying bold, italic, and other supported text styling.
- Adjusting text positions.
- Changing text dimensions.
- Adjusting alignment.
- Adjusting spacing.
- Correcting overflow.
- Other supported design adjustments.

The user must retain creative control over the final designs.

Stage 22: Export the final publication

Once the user has reviewed and edited the populated pages, the application should allow the user to export the finished work using the supported export formats.

The final output should reflect the user's approved placeholders, revised spreadsheet content, and subsequent design edits.

The export must produce real downloadable files, not just a preview or a success message.

---

6. Accuracy and document-structure detection requirements

Accuracy is a major requirement.

A mistake in identifying a style, grouping content, or assigning a page can affect an entire publication. The application must therefore use a reliable detection and validation process.

The goal is not simply to extract text. The application must understand the document's structure well enough to create the correct spreadsheet.

A. Use DOCX structural information

The application should inspect the actual DOCX/Open XML structure.

It should use the document's structural information as the primary source for:

- Paragraph styles.
- Style identifiers.
- Style definitions.

- Style inheritance.
- Paragraphs and text runs.
- Character formatting.
- Numbering and list definitions.
- Page breaks.
- Section breaks.
- Tables.
- Other supported document structures.

It must not rely exclusively on visual guesses such as font size, boldness, or text appearance.

B. Distinguish different types of formatting

The application must distinguish:

- Paragraph styles.
- Character styles.
- Direct character formatting.
- Paragraph-level formatting.
- List numbering.
- Table formatting.

For example, bold text within a Normal Text block does not automatically make the entire block a Heading 1.

Likewise, a paragraph beginning with a number does not automatically mean it is a numbered list. The application should inspect the actual numbering information where available.

C. Recognize custom styles

The application should identify custom styles and style inheritance where possible.

It must not force an unknown custom style into an unrelated recognized category merely because the visual appearance seems similar.

If the application cannot confidently classify a style, it should flag it for review.

D. Preserve document content

The application should account for relevant content in supported document structures, including:

- Main document paragraphs.
- Tables.
- Text boxes and drawing text.
- Headers and footers.

- Footnotes and endnotes.
- Other supported text containers.

It must distinguish supplementary content from the main page content where appropriate.

Unsupported structures must be reported rather than silently discarded.

E. Validate page assignment

The application must account for explicit page breaks and section breaks.

Where rendered pagination is required, it should use a reliable rendering or pagination method.

Different document editors, fonts, and rendering environments may produce different pagination. The application must not silently claim that page assignment is certain when it cannot establish the page boundaries reliably.

F. Use a multi-stage detection process

The recommended detection and validation process is:

1. Structural inspection.
2. Content extraction.
3. Style classification.
4. Occurrence grouping.
5. Page assignment.
6. Spreadsheet column calculation.
7. Spreadsheet generation.
8. Independent validation.
9. Visual verification using rendered pages where appropriate.

Visual verification should complement structural parsing, not replace it.

G. Completeness checks

Before accepting the generated spreadsheet, the application should verify that:

- Every supported content block has been processed.
- Every extracted block has an identifiable style or an explicit exception.
- No content has been accidentally omitted.
- No content has been accidentally duplicated.
- Consecutive same-style content has been grouped correctly.
- Occurrence numbers are correct.
- Every content group belongs to the correct page.
- The shared columns accommodate the maximum occurrence count for each style.

- Every document page has one corresponding data row.
- Empty cells represent genuine absence of content.
- Extraction failures have not been silently treated as empty cells.

H. Structural validation

The application should check that:

- Document page count equals spreadsheet data row count.
- Template count equals document page count.
- Template count equals spreadsheet data row count.
- Generated columns match the detected styles and their maximum occurrence counts.
- Every nonempty spreadsheet cell corresponds to valid extracted or user-revised content.
- Every content group has an identifiable page, style, and occurrence.
- No unsupported or uncertain structure is silently ignored.

I. Detect uncertainty and stop unsafe processing

The application should identify problems such as:

- Unknown custom styles.
- Conflicting style information.
- Uncertain page boundaries.
- Failed content extraction.
- Missing content blocks.
- Ambiguous column matching.
- Unexpected spreadsheet structure.
- A mismatch between document pages, spreadsheet rows, and templates.

It should not invent arbitrary confidence percentages.

When a structural decision cannot be made reliably, the application should report the problem and request review instead of silently guessing.

J. Validate at every important stage

Validation should occur at:

- DOCX inspection.
- Content extraction.
- Style classification.
- Occurrence grouping.
- Page assignment.
- Spreadsheet generation.
- Revised spreadsheet upload.

- Revised spreadsheet validation.
- Placeholder preparation.
- Placeholder review.
- Final population.

A successful earlier stage does not automatically mean that a later stage is correct.

---

7. Important rules that must never be violated

1. Do not replace the existing application with a new one.
2. Do not break the existing image workflow.
3. Do not force users to select Single Design or Bulk Design before uploading.
4. Do not automatically include all 27 style categories in every spreadsheet.
5. Do not introduce style names that were never agreed upon.
6. The exact style name is **Normal Text**.
7. Do not treat every paragraph or sentence as a separate occurrence.
8. Do not merge consecutive same-style content across different pages.
9. Do not create separate spreadsheet columns for each page.
10. Do not impose an arbitrary occurrence limit.
11. Do not create unnecessary columns or rows.
12. Do not fill genuinely empty cells with invented content.
13. Do not silently treat extraction failures as empty cells.
14. Do not lose the original page order.
15. Do not perform final placeholder matching before the revised spreadsheet is uploaded.
16. Do not populate templates immediately after revised spreadsheet upload.
17. Do not treat spreadsheet upload as user approval.
18. Do not insert actual spreadsheet content before placeholder approval.
19. Do not ignore legitimate spreadsheet changes.
20. Do not overwrite the user’s revised content with the original extraction.
21. Do not silently guess when structural information is ambiguous.
22. Do not shrink text to unreadable sizes to hide overflow.
23. Do not silently skip, duplicate, or invent pages when counts do not match.
24. Do not claim a file was downloaded when no real file was generated.
25. Preserve the user’s ability to edit and control the final design.

---

8. Implementation and testing instructions

Before implementing these requirements, inspect the existing application's architecture and determine how the new document workflow can be integrated without breaking existing functionality.

Choose a reliable DOCX/Open XML parsing approach and a validation method compatible with the application's current technology.

Use deterministic document parsing for structural facts. AI may assist with ambiguous or custom-style interpretation, but it must not replace reliable structural inspection.

Accuracy takes priority over processing speed.

Test the document workflow with:

- Documents containing all 27 recognized styles.
- Documents containing only a few of the recognized styles.
- Documents with multiple occurrences of the same style on one page.
- Documents with different occurrence counts on different pages.
- Consecutive same-style content.
- Alternating styles.
- Lists and numbering.
- Custom styles.
- Mixed character formatting.
- Explicit page breaks.
- Section breaks.
- Tables.
- Different page structures.
- Google Docs-exported DOCX files.
- Microsoft Word-created DOCX files.
- Revised spreadsheets with added, removed, renamed, or reorganized columns.
- Empty cells.
- Mismatched page, row, and template counts.
- Content that exceeds the available template space.

Verify that the generated spreadsheet, revised spreadsheet, placeholder preparation, approval, population, and export stages all work as separate, correctly ordered operations.

Do not consider the workflow complete merely because the interface looks correct. Test the actual processing and downloadable outputs.

9. Final expected document workflow

The complete sequence is:

1. User uploads DOCX.

2. Application scans the entire document.
3. Application identifies the styles actually used.
4. Application calculates each style's maximum occurrence count per page.
5. Application groups consecutive content using the same style.
6. Application assigns content to the correct pages.
7. Application generates dynamic shared spreadsheet columns.
8. Application creates one data row per document page.
9. Application validates the generated spreadsheet.
10. User downloads the actual Excel file.
11. Application records the original spreadsheet structure in the project background.
12. User uploads template images into the same project.
13. Application retains the templates and their upload order without preparing final placeholders.
14. User reviews and revises the downloaded spreadsheet.
15. User uploads the revised spreadsheet into the same project, retaining the same filename.
16. Application validates the revised spreadsheet and identifies structural changes.
17. Application updates the required placeholder structure.
18. Application prepares placeholders on the corresponding templates.
19. User reviews and edits the placeholders.
20. User explicitly approves the placeholders.
21. Application populates them using the revised spreadsheet.
22. User edits the populated pages.
23. User exports the final publication.

The central rule is that the revised spreadsheet must be uploaded and validated before final placeholder preparation. The user must then review and approve the prepared placeholders before the application inserts the actual content.

This sequence must remain intact.

LITDOM BATCH STUDIO

Functional Requirements and Application Rules

Working Specification – Version 1

1. Purpose and Core Principle

Litdom Batch Studio is a mobile-friendly bulk design and publishing application.

Its purpose is to make it possible to create and edit multiple designs efficiently without sacrificing quality, originality or creative control.

The application will initially be developed for private use. It does not need public user registration, multiple-user profiles, notifications or other unnecessary multi-user infrastructure.

The application must:

- Support three distinct workflows: Document, Image and Spreadsheet.
- Bring all three workflows into one universal editing interface.
- Accept different image and document dimensions.
- Support bulk creation and individual editing.
- Preserve user control over content, layout and export.
- Prioritise accuracy, reliability and quality.
- Avoid unnecessary complexity, paid dependencies and features that could compromise the core application.

2. Application Workflows

There are exactly three main workflows.

A. Document Workflow

DOCX → Initial Spreadsheet → Templates → Revised Spreadsheet → Placeholder Preparation → User Approval → Content Population → Editing → Export.

B. Image Workflow

Images → Automatic Placeholders → Layout Design → Apply Layout to All → Spreadsheet Upload → Automatic Population → Editing → Export.

C. Spreadsheet Workflow

Spreadsheet → Select Existing App Template → Automatic Population → Editing → Download.

All three workflows lead to the same universal editing interface.

The editor should not become three different applications. The workflow determines what content and designs are loaded into the editor, not which editor the user receives.

---

3. Dashboard

3.1 Header

The dashboard title is:

Litdom Batch Studios

The notification bell, settings icon and user profile icon are not required for the initial version.

3.2 Search

The dashboard includes a search field labelled:

Search here

Its primary purpose is searching for existing projects and designs.

3.3 Create a Project

The dashboard contains three project-entry options:

Option	Action
Document	Upload DOC
Image	Upload Images
Sheet	Upload Sheet

These options correspond to the three application workflows.

The application must support one or multiple uploaded files where appropriate. It should determine the number of files automatically rather than requiring the user to choose between single-design and bulk-design modes.

3.4 Your Projects

The dashboard contains a section titled:

Your Projects

Each project card displays:

- A preview image representing the project.
- A project icon at the top-left corner.
- A three-bar menu at the top-right corner.
- The project title underneath the preview image.

The three-bar menu has exactly three actions:

Delete

Deletes the entire project and its associated stored data, freeing the storage space occupied by that project.

Edit

Opens the universal editing interface shown on page 2 of the visual reference.

The user can edit the project and subsequently download the edited version.

Download

Opens the download interface shown on page 6 of the visual reference.

Every download must use the latest version of the project, including changes made after previous downloads.

The application must not silently reuse an old export, falsely report a successful download or download an outdated version.

3.5 Storage

The dashboard reference includes storage-used and storage-available information.

The storage display should reflect actual application storage usage. Deleting a project should free its associated storage and update the display.

The exact storage-management implementation remains subject to technical review.

---

4. Universal Editing Interface

The editor is shared by all three workflows.

The visual reference identifies the main editing controls:

- Text
- Sync Layout
- Upload
- Fields
- Add
- More

Additional controls include Undo, Redo and Home.

The editor must be optimised for mobile-phone use. Its arrangement should follow the supplied visual reference as closely as practical, with smooth operation and no unnecessary interface complexity.

The design canvas must adapt to the dimensions of the uploaded content. It must not impose one fixed design size.

The user must be able to edit designs and their content before exporting them.

4.1 Selected-Text Controls

When the user selects a text element, the editing menu changes to display relevant text-editing controls.

The reference identifies:

- Edit
- Text Styles
- Font
- Font Size
- Color
- Effects
- Format

The editor should make these controls available when text is selected rather than displaying irrelevant controls at all times.

4.2 Add Panel

The Add panel contains:

- Add Table of Content
- Add Text Box
- Default Text Styles
- Add a Heading
- Add Subheading
- Add Some Body Text
- Add Page Number
- Add Horizontal Line
- Font Combinations

These are the controls identified in the visual reference. Their detailed individual behaviour can be specified as the relevant editor sections are discussed.

4.3 Upload Panel

The Upload panel allows the user to upload images from their device for use in the design.

4.4 Fields Panel

The Fields panel is used to control and add spreadsheet fields for matching and populating designs.

4.5 Sync Layout

The Sync Layout feature allows the user to synchronise a design's layout.

The available scope options are:

- This Page
- All
- Custom

It also includes controls for synchronising a text field's layout, selecting a field and choosing what to synchronise with.

The detailed behaviour of individual synchronisation options can be specified further.

4.6 More Menu

The visual reference identifies the following items:

- Your Design’s Title
- Saved feature
- See All Pages
- Branding
- Resize Design
- Delete
- Version History
- Show Thumbnails
- Add Charts
- Add from Projects
- Templates

Their detailed behaviours remain subject to the user’s subsequent explanations.

---

5. Document Workflow: DOCX Processing

5.1 Accepted Input

The document workflow accepts DOCX files.

The DOCX may originate from Microsoft Word or Google Docs, provided the document’s style and formatting metadata are preserved.

The application must not rely solely on visual appearance to identify document styles.

5.2 Initial Style Recognition Categories

The application recognises the following 27 initial style categories:

1. Title
2. Subtitle
3. Heading 1
4. Heading 2
5. Heading 3
6. Heading 4
7. Heading 5
8. Heading 6
9. Heading 7
10. Heading 8
11. Heading 9

12. Normal Text
13. Quote
14. Intense Quote
15. Block Text
16. Body Text
17. Body Text 2
18. Body Text 3
19. Body Text First Indent
20. Body Text Indent
21. Caption
22. Book Title
23. List Paragraph
24. List Number
25. List Bullet
26. Note Heading
27. Plain Text

Important: The exact style name is Normal Text. It must not be renamed to Normal Text Paragraph.

Only styles actually used in the uploaded document should generate spreadsheet columns. The application must not automatically include all 27 categories when some are absent.

Unknown custom styles must be identified and flagged rather than silently misclassified.

Character formatting and paragraph styles must be distinguished.

5.3 Whole-Document Scanning

The application must scan the entire document before generating the spreadsheet.

It must count how many separate occurrences of each style appear on each page.

The number of shared spreadsheet columns for each style is determined by the maximum number of occurrences of that style on any single page.

Example:

Style	Page 1	Page 2	Page 3	Required columns

Heading 1	3	2	1	3
Heading 2	2	4	1	4
Normal Text	3	2	5	5
Quote	5	3	2	5

There must be enough shared columns to accommodate the maximum occurrence count.

There must not be arbitrary occurrence limits.

5.4 Occurrence Grouping

Consecutive content with the same style belongs to one occurrence.

A different style interrupts that occurrence.

Example:

Normal Text, Normal Text, Heading 1, Normal Text, Normal Text, Quote, Normal Text.

The spreadsheet should represent these as:

- Normal Text - 1: first two consecutive Normal Text paragraphs.
- Heading 1 - 1.
- Normal Text - 2: next two consecutive Normal Text paragraphs.
- Quote - 1.
- Normal Text - 3.

The application must:

- Preserve original paragraph boundaries.
- Preserve reading order.
- Avoid merging separate occurrences.
- Avoid merging content across page boundaries.
- Preserve the original style names.

5.5 Spreadsheet Structure

The generated spreadsheet has:

- Columns representing detected styles and their required occurrences.
- Rows representing document pages.

The first row contains column headers.

Page 1 occupies row 2.

Page 100 occupies row 101.

The application must preserve the document's actual page structure. It must not assume that one paragraph equals one page.

5.6 Empty Cells and Missing Content

An empty spreadsheet cell means that the corresponding content does not exist on that page.

The application must not:

- Invent text.
- Insert placeholder text into empty cells.
- Treat genuine absence as an extraction failure.
- Create text elements later for genuinely empty cells.

Extraction failures must be detected and reported separately.

5.7 Spreadsheet Generation

The application generates an actual Excel spreadsheet that the user can download on a mobile device and edit in Google Sheets.

The user must be able to:

- Correct extraction errors.
- Add missing content.
- Modify existing content.
- Add or remove columns.
- Rename columns.
- Reorganise columns.
- Expand content with additional research, examples, experience, technical details and updates.

The application must not assume that the initially generated spreadsheet is the final version.

---

6. Document Workflow: Templates and Revised Spreadsheet

6.1 Project Record

The application retains a background record of the original spreadsheet's column headers and associated project information.

This record helps identify structural changes when the revised spreadsheet is uploaded.

It must not override the user's revised spreadsheet.

The revised spreadsheet is authoritative.

6.2 Template Upload

After the initial spreadsheet has been generated and downloaded, the user uploads the design templates.

Templates are uploaded individually and stored in the project in their original order.

The application must not prepare final placeholders or perform final column matching at this stage.

The user must first review and revise the initial spreadsheet.

6.3 Revised Spreadsheet Upload

The user edits the same generated Excel file in Google Sheets and uploads the revised version into the same project.

Retaining the same filename is useful but must not be the only method of identifying the file.

The application should use project association and spreadsheet structure validation.

Legitimate structural changes are allowed, including:

- Adding columns.

- Removing columns.
- Renaming columns.
- Reorganising columns.
- Editing or expanding content.

Ambiguous changes must be flagged rather than guessed.

6.4 Correct Processing Sequence

The required sequence is:

1. Generate the initial spreadsheet.
2. Allow the user to download it.
3. Record the original spreadsheet structure in the project.
4. Upload and retain the design templates.
5. Allow the user to revise the spreadsheet.
6. Upload the revised spreadsheet into the same project.
7. Validate and process the revised spreadsheet.
8. Identify structural changes.
9. Update placeholder requirements.
10. Prepare placeholders on the corresponding templates.
11. Allow the user to review the prepared placeholders.
12. Allow the user to edit placeholder layouts.
13. Obtain the user's approval.
14. Populate the templates using the revised spreadsheet.
15. Allow the user to edit the populated designs.
16. Export the final publication.

Uploading the revised spreadsheet does not constitute approval.

Placeholder preparation, placeholder review and content population are separate stages.

6.5 Placeholder Preparation

Placeholder preparation is based on:

- Spreadsheet columns: determine the placeholder types and occurrence counts.
- Spreadsheet rows: determine which page receives the content.
- Template images: determine the available design area and placement.

Empty spreadsheet cells must not produce placeholders.

The application must consider:

- Word count.
- Character count.
- Paragraph count.
- Content hierarchy.
- Available dimensions.
- Text overflow.
- Readability.

The application must not automatically shrink text to unreadable sizes to force content into a design.

Overflow must be identified and presented for correction.

6.6 Placeholder Review and Approval

The user must be able to inspect and edit prepared placeholders before approval.

After approval, the application populates the templates using the revised spreadsheet.

The user must still be able to edit the populated designs afterward.

6.7 Content Population

The application maps each spreadsheet row to its corresponding page and each spreadsheet column to its corresponding placeholder.

It must:

- Preserve reading order.
- Preserve paragraph boundaries.
- Preserve content hierarchy.
- Leave genuinely empty cells empty.
- Avoid duplicating or skipping content.
- Avoid using outdated spreadsheet content.

6.8 Validation

The number of spreadsheet data rows, templates and document pages must be consistent.

If they do not match, the application must report the mismatch.

It must not silently skip pages, duplicate templates or invent missing data.

If the revised spreadsheet has no structural changes, the application should retain the existing placeholder structure instead of rebuilding or duplicating it unnecessarily.

If columns change, placeholder requirements must be updated accordingly.

---

7. Image Workflow

The image workflow accepts one or multiple uploaded images.

The application must accept images of arbitrary dimensions.

It must not reject images because they do not match Pinterest, Instagram, LinkedIn or any other predefined platform dimensions.

7.1 Automatic Placeholders

After uploading images, the application automatically adds standard text placeholders.

The user arranges the layout on the first image.

The user can then select:

Apply Layout to All

The application applies that layout to all uploaded images.

7.2 Spreadsheet Population

The user uploads a spreadsheet.

Spreadsheet columns correspond to the available placeholders.

Spreadsheet rows correspond to the uploaded images.

The application automatically populates the images using the spreadsheet data.

The user can then edit:

- Text.
- Font.
- Font size.
- Colour.

- Effects.
- Position.
- Dimensions.
- Alignment.
- Spacing.
- Overflow.
- Other available design properties.

The completed images are exported through the universal editor.

---

8. Spreadsheet Workflow

The spreadsheet workflow allows the user to upload a spreadsheet and select an existing application template.

The application uses the selected template's predefined columns to populate the design automatically.

The sequence is:

1. Upload spreadsheet.
2. Select an existing application template.
3. Match the spreadsheet data to the template.
4. Automatically populate the designs.
5. Open the universal editor.
6. Allow the user to make edits.
7. Download the completed designs.

The spreadsheet must follow the selected template's predefined column structure.

---

9. Download System

The download system must generate fresh exports using the latest project content and settings.

It must not reuse outdated exports after the user edits a project.

The download interface contains a file-type selection box with a dropdown arrow. Tapping the box or arrow opens the available file formats.

The interface changes depending on the selected format.

9.1 Supported Core File Types

Image formats

- PNG
- JPG
- JPEG
- WebP

Document format

- PDF

Optional formats for technical assessment

- HTML
- PPTX

HTML could be useful for converting a design into a functional web interface, but it requires more than ordinary image export. It may require HTML, CSS and JavaScript generation.

PPTX could be useful for presentations with automatic slide transitions, potentially every two seconds.

These two formats are not confirmed core requirements. The technical reviewer should assess their complexity and whether they can be implemented without compromising the main application.

The core export formats take priority.

---

10. Image Export Interface

When PNG, JPG, JPEG or WebP is selected, the image-export interface is displayed.

The image-export interface contains:

- Size
- Edit
- Quality

- Limit
- Select Page
- Download

10.1 Size

The displayed size represents the actual dimensions of the current design.

It must not impose a fixed platform-specific dimension.

The application must accept different image dimensions, including square, landscape, portrait, long, narrow and custom-sized images.

10.2 Edit / Resize

The Edit option opens a resizing interface.

It contains:

- Enter Width.
- Enter Height.
- Save.

The user can enter custom dimensions.

After saving, the application must recalculate the layout and adapt text, fonts, images and other design elements to the new dimensions.

The user must be able to preview the resized design before downloading it.

The exported version must use the new dimensions and adjusted layout, not the original version.

The resizing process must preserve readability and design coherence rather than merely stretching the design.

10.3 Quality

Three options are available:

Low: Produces a smaller, lower-quality image.

Medium: Produces normal image quality.

High: Produces exceptionally clean and crisp output, with clearly rendered fonts, backgrounds and design elements.

High quality should aim for professional-looking output comparable to polished design software.

10.4 Limit / Compression

The Limit option allows the user to specify a target file size when compressing exports, particularly bulk image exports.

The application must evaluate the number of images, their dimensions and content before accepting a compression target.

It must establish a sensible minimum size that avoids unacceptable visual degradation.

For example, if the user selects 100 images, the application must not blindly accept an unrealistic request to compress all 100 images into an extremely small file.

If the requested size is not achievable at acceptable quality, the application must explain the limitation and offer a reasonable minimum.

10.5 Image Page Selection

Three options are available:

Page + Current Page Number

Downloads only the page currently open in the editor.

For example, if the user is editing page 7, the option displays Page 7. If the user moves to page 8, it must dynamically display Page 8.

All

Downloads all designs in the project.

Select

Opens a custom selection interface containing small thumbnails of all available designs.

The user can manually select the pages to download.

Only selected pages are exported.

If five pages are selected, only those five pages should be downloaded.

11. PDF Export Interface

When PDF is selected as the file type, the interface changes.

Instead of the image-export Size option, the PDF interface displays:

Settings

The user presses Edit Settings to open the PDF settings pop-up.

11.1 PDF Settings

There are five settings:

1. Digital
2. RGB
3. CMYK
4. Flatten
5. Crop Marks & Bleed

11.2 Default Settings

The default configuration is:

Setting	Default
Digital	ON
RGB	OFF
CMYK	OFF

Flatten	OFF
Crop Marks & Bleed	OFF

Digital is the default PDF export mode.

11.3 Colour Mode Rules

Digital, RGB and CMYK are mutually exclusive.

Only one of these three options may be ON at a time.

Examples:

- Digital ON means RGB and CMYK must be OFF.
- RGB ON means Digital and CMYK must be OFF.
- CMYK ON means Digital and RGB must be OFF.

The user must turn off the current mode before selecting another.

If the application encounters a configuration with two or more of these modes enabled, it must prevent the export and display:

Error: Please choose only one colour mode.

CMYK is mandatory

The application must genuinely support CMYK PDF output.

It must not simply label an RGB PDF as CMYK.

The exported PDF must contain actual CMYK colour output, and this must be verified technically.

If a technical limitation prevents correct CMYK conversion, the application must report it rather than silently generate an incorrect file.

11.4 Flatten

Flatten is an independent toggle.

When enabled, the PDF is flattened for print production.

11.5 Crop Marks & Bleed

Crop Marks & Bleed is another independent toggle.

When enabled, the PDF includes appropriate crop marks and bleed for printing.

Flatten and Crop Marks & Bleed may both be enabled simultaneously.

They may also be enabled or disabled independently.

11.6 PDF Page Selection

The same three page-selection options are available:

- Page + Current Page Number.
- All.
- Select.

Page + Current Page Number exports only the current page.

All exports every page.

Select opens the custom thumbnail-selection interface, allowing the user to choose specific pages.

11.7 PDF Download

When the user presses Download, the application must generate the PDF using:

- The latest project version.
- The selected colour mode.
- The Flatten setting.
- The Crop Marks & Bleed setting.
- The selected pages.

The export must not reuse an earlier PDF generated before the latest edits or settings changes.

---

12. Pop-up Behaviour

Every pop-up should have a small Cancel button at the top.

Tapping outside a pop-up may also close it.

However, the visible Cancel button should always be available.

---

13. Project Storage and Export Reliability

The application must maintain reliable project data and storage management.

Deleting a project should delete its associated stored data and free the corresponding storage.

Editing and downloading must not corrupt existing project content.

Every export action must use the latest project state.

The application must verify that a new export has actually been generated before reporting success.

An earlier downloaded file must never be mistaken for the latest export.

---

14. Document Processing Accuracy and Technical Requirements

The document workflow must prioritise structural accuracy.

The application should use DOCX/Open XML structure as its primary source of truth, rather than relying exclusively on visual or AI-based interpretation.

The technical implementation should inspect supported document structures, including:

- Paragraphs.
- Style references.
- Style definitions and inheritance.
- Text runs.
- Character formatting.
- Direct formatting.
- Numbering definitions.
- Page and section breaks.

- Tables.
- Text boxes and drawing text.
- Headers and footers.
- Footnotes and endnotes.
- Other supported text containers.

The application must distinguish paragraph styles, character styles, direct formatting, table styles and numbering.

It must not classify a paragraph as a numbered list simply because its text begins with a number.

Unknown custom styles, conflicting classifications, uncertain pagination and extraction failures must be flagged.

Page assignment must preserve actual document structure. Explicit breaks and reliable rendering or pagination methods should be considered where necessary.

Different rendering engines, fonts and software may produce different page boundaries. Uncertainty must be reported rather than concealed.

14.1 Processing Pipeline

The application should use a staged processing and validation pipeline:

1. Structural document inspection.
2. Content extraction.
3. Style classification.
4. Occurrence grouping.
5. Page assignment.
6. Spreadsheet calculation.
7. Independent validation.
8. Visual verification where necessary.

14.2 Completeness Checks

The application should verify that:

- Every supported paragraph has been processed.
- Every text block has a valid style classification or an explicit exception.
- No content has been duplicated or omitted.
- Occurrence groups are correct.
- Page assignments are correct.
- Enough shared columns have been generated.

- Every document page has a corresponding spreadsheet row.
- Empty cells represent genuine absence.
- Extraction failures are flagged.

14.3 Validation and Error Handling

Validation checkpoints should exist at:

- DOCX inspection.
- Content extraction.
- Style classification.
- Occurrence grouping.
- Page assignment.
- Spreadsheet generation.
- Revised spreadsheet validation.
- Placeholder preparation.
- Placeholder review.
- Final content population.

The application must stop unsafe processing when it encounters unresolved issues such as:

- Uncertain page boundaries.
- Missing content blocks.
- Conflicting classifications.
- Unresolved custom styles.
- Row/page mismatches.
- Unexpected spreadsheet structures.
- Ambiguous matching.

The user must be informed of the issue and given an opportunity to correct it.

14.4 Testing

The document-processing system should be tested with:

- All 27 recognised styles.
- Documents containing only a subset of styles.
- Repeated styles.
- Consecutive same-style paragraphs.
- Alternating styles.
- Lists and numbering.
- Custom styles.
- Mixed formatting.
- Page breaks.

- Section breaks.
- Tables.
- Different page structures.
- DOCX files originating from Microsoft Word.
- DOCX files originating from Google Docs.

Accuracy is more important than processing speed.

---

15. Visual Reference Requirements

The supplied visual reference PDF contains screenshots of the intended application interface.

The application should reproduce the actual Litdom Batch Studio interface as closely as practical, especially on mobile devices.

The reference includes dashboard, editor, selected-text controls, Add, Upload, Fields, Sync Layout, spreadsheet upload, More menu and download interfaces.

Important Visual Reference Clarification:

The blue area in the original screenshot belongs to Canva's interface and was captured accidentally. It is not part of Litdom Batch Studio.

Do not reproduce the blue area, use its colours or interpret its elements as application controls.

Only reproduce the actual application interface surrounding it.

The visual PDF is a reference for interface appearance. This functional specification defines the confirmed application behaviour.

If a visual detail and a confirmed functional rule appear to conflict, do not silently change the requirement. Identify the conflict for review.

---

16. Technical Review and Implementation Instructions

The application already exists and has been developed using Google AI Studio and GitHub Pages.

The existing working application must be inspected before implementation changes are made.

The developer or AI implementation system must:

1. Examine the existing application and its current features.
2. Review the complete functional specification.
3. Review the visual reference PDF.
4. Identify technical limitations and missing implementation details.
5. Suggest suitable technical approaches.
6. Preserve all confirmed requirements.
7. Avoid unnecessary infrastructure and complexity.
8. Implement changes incrementally.
9. Test existing and newly implemented functionality.
10. Avoid breaking features that already work.

The technical reviewer may suggest improvements, but must distinguish suggestions from confirmed requirements.

It must not remove, reinterpret or replace a confirmed requirement without first explaining the issue and obtaining approval.

The initial version is for private use. Avoid unnecessary public-facing infrastructure.

Prioritise free or low-cost technologies and avoid payment-dependent features wherever reasonably possible.

CMYK PDF export, accurate document processing, reliable spreadsheet handling, arbitrary image dimensions, mobile usability and fresh export generation are important requirements that must not be casually omitted.

---

17. Features Still Requiring Detailed Explanation

The following areas have been identified in the interface or earlier requirements but need further explanation before the specification is considered complete:

- Detailed behaviour of Undo, Redo and Home.
- Detailed behaviour of Text and Text Styles.
- Detailed behaviour of Branding.
- Detailed behaviour of Version History.
- Detailed behaviour of Show Thumbnails.

- Detailed behaviour of Add Charts.
- Detailed behaviour of Add from Projects.
- Detailed behaviour of Templates.
- Detailed behaviour of See All Pages.
- Detailed behaviour of Save.
- Detailed behaviour of Delete within the editor.
- Detailed behaviour of the spreadsheet upload and matching interface.
- Detailed behaviour of project storage and any remaining dashboard elements.
- Whether HTML and PPTX exports are technically worthwhile.

These should be documented as the user explains them.

End of Working Specification – Version 1

Litdom Batch Studio — Functional Specification

Version 2: Storage, Editor Navigation, Social Publishing and Workflow Extensions

Continuation of Version 1. This document adds the newly explained functionality. Previously documented requirements remain in force and are not replaced.

18. Storage Management

The application must display actual storage usage.

For example:

- Total storage: 1.25 GB
- Used: 250 MB
- Remaining: approximately 1 GB

These are illustrative figures, not fixed values.

The application must calculate actual storage usage, update the figures when files are added or deleted, and genuinely release associated storage when a project is deleted.

The displayed figures must never be updated independently of the actual stored files.

Storage management must account for potentially large projects containing hundreds or thousands of images.

19. Transition from Document Workflow to Editor

The document workflow proceeds as follows:

1. Upload DOCX.
2. Scan the document.
3. Generate the initial spreadsheet.
4. User reviews and revises the spreadsheet.
5. Upload matching background templates.
6. Upload the revised spreadsheet.
7. Analyse the spreadsheet and templates.
8. Generate the required placeholders automatically.
9. User reviews the arrangement.
10. User initiates Match and Populate.
11. Enter the universal editor for final adjustments.

The editor is shared by all three workflows. Its behaviour depends on the workflow from which the user entered it.

20. Editor Navigation and History Controls

The editor's top navigation contains:

- Home
- Redo
- Undo
- Three-bar design navigation
- Downward-pointing arrow
- Upward-pointing arrow

Exact positioning may be adjusted during implementation, provided the functions remain accessible and mobile-friendly.

20.1 Undo

Undo must reverse editing actions sequentially.

Every individual action, including small movements and adjustments, must be recorded.

For example, if an element is moved right three times, three Undo actions must reverse those movements individually.

20.2 Redo

Redo restores actions that were undone, in their original sequence.

20.3 Three-bar Design Navigation

The editor's three-bar control displays all designs in the current project as small thumbnails.

- Thumbnails appear side by side.
- The user can scroll horizontally.
- Selecting a thumbnail opens that design in the main editor.
- Users must be able to navigate large collections without opening every design sequentially.

20.4 Downward Arrow: Download

The downward arrow opens the download interface.

The user selects the required format, settings and pages before downloading.

This is separate from social publishing.

21. Dashboard Project Card Menu

Each project preview card on the Dashboard (page 1) has a three-bar menu at its top-right corner.

This menu must contain exactly four actions:

1. **Download:** Opens the project's download interface.
2. **Edit:** Opens the project in the universal editor.
3. **Delete:** Deletes the entire project and its associated stored files, releasing the corresponding storage.
4. **Save as Template:** Saves the selected design as a reusable template.

The Save as Template action belongs specifically to this Dashboard project-card menu. It must not be placed in the editor's three-bar navigation menu.

21.1 Save as Template

When the user selects Save as Template, the application saves the selected design's complete reusable structure:

- Background image.
- Existing placeholders.
- Placeholder field names and identities.
- Placeholder positions.
- Placeholder dimensions.
- Text formatting and relevant layout properties.
- Placeholder arrangement and order.

The saved template must preserve both the background and its associated placeholders.

Saving only the background image is insufficient.

The saved template must become available for selection in future Upload Spreadsheet workflows.

The application must provide a way to manage saved templates, including removing templates that are no longer needed.

Saved templates must persist beyond the current editing session. They should be stored independently of the temporary editing state so that deleting an original project does not unintentionally delete a template that has already been saved for reuse.

22. Project Card Opening Behaviour

Users can open a project through either:

1. Selecting the project card itself.
2. Opening its three-bar menu and selecting Edit.

Both actions lead to the project's editing interface.

23. Upward Arrow: Social Media Publishing

The upward-pointing arrow provides a social-media handoff function.

It must remain separate from the download function.

The confirmed destination platforms are:

1. Pinterest
2. LinkedIn
3. X (formerly Twitter)
4. WhatsApp
5. Substack
6. Medium
7. Reddit
8. Discord
9. Instagram
10. Facebook
11. Threads
12. Tumblr

TikTok image publishing is excluded for now. It is reserved for a future video workflow.

23.1 Design Selection

Before transferring designs, the user selects which designs to publish.

Available selection modes:

- Current Page
- All
- Custom

Custom opens the thumbnail selection interface so the user can choose individual designs.

Only selected designs are transferred.

23.2 Platform Selection and Transfer

After selecting designs, the user chooses a destination platform.

The application hands the selected designs to the platform's supported creation, sharing or upload interface.

For example:

1. Select a design.
2. Press the upward arrow.
3. Choose Pinterest.
4. The application transfers the design through the supported integration.
5. Pinterest opens with the design ready for the remaining publishing steps.

The application must use actual supported platform capabilities. It must not claim a successful transfer when the destination has not received the content.

24. The Three Production Workflows: Shared Editor, Different Behaviour

All three workflows lead to the same universal editor. However, their background sources, placeholder-generation processes and spreadsheet-matching behaviour differ.

24.1 Upload Images Workflow

The user supplies the background images.

Sequence:

1. Upload one or multiple images.
2. The system automatically places the standard text placeholders on the uploaded images.

3. The user can adjust the arrangement.
4. The user presses the Plus (+) control and uploads a spreadsheet.
5. The spreadsheet columns must match the placeholder fields.
6. The system automatically detects and matches the columns.
7. The system populates the corresponding placeholders.

In this workflow, placeholders exist before spreadsheet upload.

The Fields control remains available for adding additional fields when required.

For example, a spreadsheet may contain six columns while a particular image initially has only one placeholder. The user can use Fields to add the required additional placeholders to the design.

24.2 Upload Document Workflow

The user supplies a DOCX document and matching background images.

The document is scanned using the 27 recognised Word formatting styles already defined in Version 1.

The spreadsheet represents the document structure:

DOCX structure	**Spreadsheet representation**
Recognised Word formatting styles	Columns
Document pages	Rows
Actual text	Cell values
Content using a particular style	Value in that style’s column

The 27 styles provide the structural language through which the application understands the document.

Columns represent content types and corresponding placeholder types. Rows represent document pages. Cells contain the actual content.

The system must use this information to determine which placeholders are required on each page.

The user must not manually connect spreadsheet columns to placeholders. The system performs that entire matching process automatically.

Template count validation

The number of uploaded background templates must match the number of document pages/rows expected by the system.

For example, a 100-page document requires 100 corresponding templates.

If the user supplies only 90, the application must identify the mismatch and explain it rather than silently treating the workflow as complete.

Automatic placeholder generation

After the user uploads the revised spreadsheet through the Plus (+) control:

1. The system analyses the spreadsheet.
2. It detects the fields.
3. It determines which columns contain content in each row.
4. It creates the corresponding placeholders on each matching background template.
5. It preserves the spreadsheet's left-to-right column order.

Every spreadsheet column represents a placeholder type.

For example:

Header	Subtitle	Title	Body Text
Welcome	Introduction		
Chapter One		The Beginning	Main content

Template 1 receives Header followed by Subtitle.

Template 2 receives Header, Title and Body Text, in that order.

Empty cells must not produce placeholders.

The system uses column identity, row occupancy and column order to establish the correct arrangement.

Match and Populate

The Plus (+) upload interface displays:

- Spreadsheet uploaded successfully.
- Analysis and fields detected successfully.
- Ready to populate.

The user can close the small upload panel using its Cancel control and inspect the generated placeholders.

The spreadsheet remains available through the Plus (+) control.

Once satisfied, the user selects Match and Populate.

The application uses the matches already established to populate the placeholders with the spreadsheet content.

The user does not manually perform column-to-placeholder matching.

24.3 Upload Spreadsheet Workflow

This workflow is for users who have spreadsheet content but do not want to supply background images or a DOCX document.

The application provides two possible background arrangements.

A. Existing system template

The application offers stored templates created from reusable designs.

Each stored template contains:

- Its background image.
- Its predefined placeholder fields.
- The positions and dimensions of those fields.
- Their formatting and arrangement.

The user uploads a spreadsheet and selects an existing template.

The application checks whether the spreadsheet columns are compatible with that template's predefined fields.

If they match, the system automatically populates the template.

If they do not match, the application identifies the incompatibility and explains which fields the selected template supports.

The user must be able to correct the spreadsheet and try again.

Saved templates created through the Dashboard's Save as Template action must also appear in this collection.

B. Create a template using the plain-white arrangement

This option does not introduce a background editor.

- The background is permanently plain white.
- The user cannot change its colour or design.
- The spreadsheet columns establish the available text fields.
- The user can edit the text and its supported formatting in the universal editor.

The purpose is straightforward content production, not graphic design.

25. Implementation Instructions

Update the existing Litdom Batch Studio application.

Do not rebuild the application unnecessarily or introduce a separate graphic-design editor.

Preserve the existing confirmed workflows and functionality.

Implement Save as Template as an action in the three-bar menu at the top-right of each Dashboard project preview card.

The menu must contain exactly four actions: Download, Edit, Delete and Save as Template.

Do not put Save as Template in the editor's three-bar navigation menu.

Use the existing design, placeholder and Fields architecture to save reusable templates.

Ensure a saved template preserves its background and all associated placeholder information.

Ensure the Upload Spreadsheet workflow can retrieve saved templates, validate spreadsheet compatibility and populate the saved fields automatically.

Saved templates must persist independently of their original projects.

Maintain mobile-first usability, reliable storage and the existing project's free/low-cost architecture.

Before changing existing functionality, inspect the current implementation and identify the appropriate integration points. Do not remove or break working features.

Litdom Batch Studio — Version 3

Text Editing, Formatting and Selection Behaviour

31. Two Distinct Levels of Text Editing

The editor must support two separate levels of text editing. They may contain similar formatting operations, but they must not be treated as the same function.

Every placeholder, added field and text box is a text element.

When a user taps a text element:

- A selection boundary appears around the text element.
- The main lower toolbar changes to the selected-text toolbar.
- This toolbar is used to modify the entire text element as a whole.

The selected-text toolbar contains:

1. Edit
2. Text Styles
3. Font
4. Font Size
5. Color
6. Effect
7. Format

The controls in this toolbar apply to the entire selected text element unless a particular control specifies otherwise.

32. Edit Button: Individual Text Selection and Formatting

The Edit button is represented by a pencil icon in the main selected-text toolbar.

When the user taps Edit, the editor must enter a separate text-editing mode.

This mode is specifically for editing the actual content inside the selected text box and applying formatting to selected words, letters or groups of words.

It must not be confused with editing the text box as a whole.

The user must be able to:

- Retype the entire content of the text box.
- Delete the existing content using the phone keyboard's Backspace key and enter new content.
- Select a particular word or phrase, delete it and replace it.
- Highlight individual letters, words, phrases or groups of words.
- Apply formatting to the highlighted selection.

The user confirms text-entry changes using an OK control.

33. Edit Toolbar Interface

When Edit is activated, display a compact, horizontal toolbar immediately above the phone's keyboard.

The toolbar must:

- Be horizontally scrollable.
- Remain compact and suitable for a mobile screen.
- Not appear as a large square pop-up or separate editing window.
- Allow the user to access the available text-editing and formatting operations while the keyboard is open.

The user must be able to use the phone's native text-selection and highlighting behaviour to select the content they want to modify.

Selection-dependent formatting must only act on the highlighted content. If nothing is selected, those operations must not modify any text.

34. Edit Toolbar Functions

The Edit toolbar must support the following operations:

1. Rewrite text.
2. Replace highlighted text.
3. Change the font of highlighted text.
4. Change the font size of highlighted text.
5. Bold highlighted text.
6. Italicise highlighted text.
7. Underline highlighted text.

8. Cancel.
9. Switch between uppercase and lowercase.
10. Mixed Formatting.
11. Change text formatting.
12. Change text styles.
13. Adjust letter spacing.
14. Adjust line spacing.
15. Adjust anchoring.
16. Change the colour of highlighted text.

The colour operation is mandatory. Users must be able to change the colour of an individual word or selected group of words without changing the colour of the rest of the text box.

35. Uppercase, Lowercase and Mixed Formatting

The uppercase/lowercase control must use the familiar symbol consisting of a capital A beside a lowercase a.

Normal tap:

- If the selected text is lowercase, convert all its letters to uppercase.
- If the selected text is uppercase, convert all its letters to lowercase.

Long press:
A long press on the same control must reveal the Mixed Formatting option.

When selected, Mixed Formatting applies title-style capitalisation:

- The first letter of each word becomes uppercase.
- The remaining letters of each word become lowercase.

Example:

```
CHANGE HIGHLIGHTED TEXT
```

becomes:

```
Change Highlighted Text
```

This operation must work on the highlighted text selection.

36. Formatting Scope: Whole Text Element Versus Highlighted Text

This distinction is a fundamental implementation requirement.

The same type of formatting operation may be available in both toolbars, but its target must differ.

Formatting operation	Main selected-text toolbar	Edit toolbar
Change font	Entire text element	Highlighted text only
Change font size	Entire text element	Highlighted text only
Change colour	Entire text element	Highlighted text only
Bold	Entire text element	Highlighted text only
Italic	Entire text element	Highlighted text only
Underline	Entire text element	Highlighted text only
Other formatting	Entire text element	Highlighted text only

Examples:

- To make an entire title gold, select its text box and use the main Color control.
- To make only one word in that title gold, select the text box, tap Edit, highlight that word and use the Edit toolbar's colour control.
- To bold an entire text element, use the main toolbar's formatting controls.
- To bold only one word, use Edit and highlight that word first.

Do not merge these two editing environments or make the Edit button simply open another version of the same whole-element controls.

37. Page-Specific Editing Rule

All text edits and formatting changes must be page-specific by default.

Editing a placeholder or field on one page must not automatically change the corresponding field on other pages.

For example:

- Changing the title on Page 1 must not change the title on Page 2.
- Replacing text in a field on one page must not replace the same field's content throughout the design.
- Adding or modifying a text element on one page must not automatically reproduce that change on other pages.

Changes remain on the current page unless the user explicitly chooses a separate operation that applies changes to multiple pages or the entire design.

Never propagate changes across pages without an explicit instruction or control authorising that behaviour.

38. Formatting Accuracy and Visual Rendering

Formatting controls must perform their actual advertised functions.

- **Bold must produce visibly bold text.** It must not merely activate a button or change an internal formatting state without changing the rendered text.
- **Italic must produce visibly italic text.**
- **Underline must produce a genuine visible underline beneath the selected text.**
- **Colour changes must visibly affect the intended text selection.**

The underline implementation requires particular attention.

Do not place the underline so close to the letters that it appears attached to the bottom of the characters or collides with their descenders.

Provide appropriate visual breathing room between the text and its underline. The underline must remain recognisable as a separate formatting line while still clearly belonging to the underlined text.

This spacing must be consistent and visually comfortable across different fonts and font sizes.

39. Implementation and Testing Requirements

Test both editing environments independently.

Verify that:

1. Selecting a text element opens the main selected-text toolbar.
2. The main toolbar modifies the entire selected text element.
3. Tapping Edit opens the separate keyboard-based editing environment.
4. The Edit toolbar appears horizontally above the keyboard.
5. Highlighting a word and applying formatting affects only that word.
6. Highlighting a phrase and changing its colour affects only that phrase.
7. Replacing selected text works correctly.
8. Uppercase, lowercase and Mixed Formatting work as described.
9. Bold, italic and underline produce visibly correct results.
10. Underline spacing remains comfortable and does not collide with letters.
11. Editing a field on one page does not alter the same field on other pages.
12. Text retains its other formatting when an unrelated portion is edited, wherever technically possible.

Do not implement the two editing environments as one undifferentiated toolbar. Their separate purposes and formatting scopes must be preserved.

Litdom Batch Studio — Version 3

Editor: Text Selection, Editing, Typography and Formatting

26. Text Elements and Selection Behaviour

Every placeholder, added field and text box in the editor is a text element.

When the user taps a text element:

- A selection boundary appears around it.
- The boundary has four corner circles and additional side controls for manipulating and resizing the text element.
- The user can adjust the text box's dimensions and position.

Resizing the text box must cause the text to reflow naturally. For example, narrowing a text box containing "Your Design Goes Here" may cause the words to wrap onto additional lines. The text content must remain intact.

When a text element is selected, the main lower toolbar changes to the selected-text toolbar.

27. Two Separate Text-Editing Environments

The editor must distinguish between:

A. Whole-text-element editing

When a user selects a text box, the main selected-text toolbar appears. Its controls apply to the entire selected text element.

B. Selected-content editing

When the user selects a text box and taps its Edit option (pencil icon), a separate editing environment opens. This environment allows the user to edit the actual content inside the text box and apply formatting to specifically highlighted letters, words or phrases.

These are two different editing environments. Do not combine them or treat them as interchangeable.

The main toolbar edits the text element as a whole. The Edit toolbar edits selected content inside that element.

28. Page-Specific Editing Rule

Every modification made to a page must remain on that page by default.

Examples:

- Editing the title on Page 1 must not change the title on Page 2.
- Changing text belonging to a spreadsheet field on one page must not change that field on other pages.
- Adding a text box to one page must not automatically add it to other pages.
- Changing the font, size, colour, effect or formatting of one page's text must not automatically change other pages.

Changes may apply across multiple pages only when the user explicitly selects a control or operation that authorises that behaviour.

Never propagate page-specific changes throughout the design automatically.

29. Main Selected-Text Toolbar

When the user taps a text element, display the following lower toolbar:

1. Edit
2. Text Styles
3. Font
4. Font Size
5. Color
6. Effect
7. Format

Except for Edit, these controls modify the entire selected text element.

30. Edit Button and Keyboard-Based Editing

The Edit button is represented by a pencil icon.

When tapped:

- Open the text-editing environment.
- Make the phone's keyboard available.
- Allow the user to select and edit text using the phone's native highlighting and selection controls.
- Display a separate, compact formatting toolbar immediately above the keyboard.

The user must be able to:

- Delete all existing content using Backspace and type replacement content.
- Select a particular word or phrase, delete it and replace it.
- Rewrite the entire text.
- Modify individual words, letters or groups of words.

An OK control must allow the user to confirm text-entry changes.

The Edit function must not automatically apply formatting to the entire text box. Formatting operations in this environment apply to the highlighted selection only.

31. Edit Toolbar Design and Functions

The Edit toolbar must be:

- Horizontally scrollable.
- Thin but clearly visible.
- Positioned immediately above the phone's keyboard.
- Compact enough not to obstruct the text being edited.
- Capable of displaying additional functions through horizontal scrolling.

Do not stack all the controls vertically or allow the toolbar to cover the editing workspace.

Its functions must include:

1. Font selection.
2. Text Styles.
3. Font size.
4. Color.
5. Bold.
6. Italic.
7. Underline.
8. Cancel.

9. Uppercase/lowercase conversion.
10. Mixed Formatting.
11. Other text formatting.
12. Letter spacing.
13. Line spacing.
14. Anchoring.

Selection-dependent operations must only work on highlighted text. If the user has not highlighted any content, those operations must not modify the text.

32. Uppercase, Lowercase and Mixed Formatting

The uppercase/lowercase control must use a capital A beside a lowercase a.

Normal tap:

- Lowercase selected text becomes entirely uppercase.
- Uppercase selected text becomes entirely lowercase.

Long press:

A long press on the same control reveals the Mixed Formatting option.

When selected, Mixed Formatting capitalises the first letter of each word and converts the remaining letters to lowercase.

Example:

```
CHANGE HIGHLIGHTED TEXT
```

becomes:

```
Change Highlighted Text
```

This operation applies only to the highlighted selection in Edit mode.

33. Text Styles

Text Styles changes the recognised style of the entire selected text element.

Examples:

- Change Title to Paragraph.
- Change Title to Heading 1.
- Change one recognised text style to another available style.

The system must use the recognised text-style categories available in the application.

The Text Styles control must open a compact selection interface from the bottom of the screen. It must not cover the entire workspace.

The change applies only to the selected text element on the current page. It must not automatically change corresponding text elements on other pages.

34. Font Management

The Font control changes the font of the entire selected text element.

The font-selection interface must support uploading custom fonts.

Custom font upload:

- Users must be able to upload font files from their devices.
- An uploaded font must automatically be added to the application's available font library.
- It must become selectable like the application's original fonts.
- Uploaded fonts must remain available in subsequent editing sessions and projects.
- Every newly uploaded font increases the available font collection.

For example:

5 original fonts + 1 uploaded font = 6 available fonts.

Another uploaded font = 7 available fonts.

Do not require users to upload the same font repeatedly for different projects.

Font weights:

Every font, including user-uploaded fonts, must provide three weight options:

- Light
- Medium
- Bold

A small downward-pointing arrow beside each font must reveal these weight options.

Selecting a weight must visibly change the appearance of the selected text.

Where the font provides genuine Light, Medium and Bold font files, use the appropriate font files. Where weights must be generated, use a reliable rendering method that produces satisfactory visual results.

Bold must look genuinely bold. Do not implement weight selection as a decorative control that changes no visible appearance.

35. Font Size

Font Size changes the size of the entire selected text element.

Provide three methods:

1. Plus (+) to increase the font size.
2. Minus (−) to decrease the font size.
3. Custom input to enter the exact font size required.

The selected text element must visibly update when the user changes its size.

36. Color and Gradient

The Color control changes the colour of the entire selected text element.

It must support:

- Standard colour selection, including colours such as pink and blue.
- Custom HEX colour codes.
- Gradient colours.

Gradient support is mandatory.

Users must be able to create multi-colour gradients in which colours blend across the text rather than displaying as one uniform colour.

Examples include:

- Blue blending into yellow.
- Yellow blending into green.
- Red blending into orange.
- Green blending into blue.

Allow users to select and combine colours for their desired gradient appearance.

The system must render genuine gradients on the text.

The scope of the colour operation depends on the editing environment:

- Main Color control: entire selected text element.
- Edit Color control: highlighted words or letters only.

37. Text Effects

The Effect control applies effects to the entire selected text element.

The effects library must provide functional text effects, including:

- Drop shadow.
- Glow.
- Hollow.
- Outline.
- Echo.
- Glitch.

These are foundational examples, not a limit on the effects library. Additional effects should be introduced progressively.

Effects must produce genuine visual results. Selecting an effect must not merely activate a button or change an internal setting without visibly changing the text.

Provide adjustable effect parameters appropriate to each effect, including:

- Intensity.
- Offset.
- Blur.
- Effect colour.
- Outline colour.
- Background colour.
- Other relevant effect-specific settings.

Functional requirements:

- Hollow must genuinely create a hollow text appearance.
- Outline must produce a visible outline around the letters.
- Glow must visibly illuminate the text.
- Drop shadow must create an actual shadow.
- Echo must create a visible echo effect.
- Glitch must produce a recognisable glitch effect.
- Changing offset must visibly move the effect.
- Changing intensity, blur or other parameters must visibly change the result.

The user must be able to adjust effect colours, including outline colour and background colour where applicable.

The effects interface must be expandable. Implement foundational effects first, then add more effects through subsequent updates without breaking existing functionality.

The long-term objective is to make typography and text effects major strengths of Litdom Batch Studio.

38. Format

Format is a separate control from Effect and applies to the entire selected text element.

It is responsible for organising and formatting the text inside the selected text box.

It must support:

1. Bold.
2. Italic.
3. Underline.
4. Text alignment.
5. Uppercase conversion.
6. Lowercase conversion.
7. Mixed Formatting.
8. Automatic text formatting.
9. Numbered lists.
10. Bulleted lists.
11. List organisation and formatting.
12. Text styles and organisation.
13. Letter spacing.
14. Line spacing.

Formatting controls must produce their actual intended results.

The scope distinction remains:

- Main Format control: formats the entire selected text element.
- Edit formatting controls: format highlighted words, letters or phrases only.

39. Formatting Accuracy and Visual Quality

Formatting controls must perform the functions their names advertise.

Bold

- Must visibly bold the text.
- Must not merely activate a button without changing the rendered font weight.

Italic

- Must visibly italicise the text.

Underline

- Must produce a genuine visible underline.
- The underline must have sufficient breathing room beneath the letters.
- Do not position it so close that it appears attached to the bottom of the characters or collides with descenders.
- Maintain comfortable, consistent spacing across different fonts and font sizes.

Color

- Must visibly change the intended text to the selected colour or gradient.

Effects

- Must render the actual selected effect.
- Effect adjustments must visibly update the result.

The overall typography and formatting experience must be reliable and visually satisfactory, not a collection of controls that only appear functional.

40. Pop-up and Panel Design Requirements

Apply the following rules to pop-ups and selection panels throughout the editor:

- Panels should emerge from the bottom of the screen or the area associated with the control being selected.
- They should occupy approximately one-quarter of the available screen.
- They must not unnecessarily cover the entire workspace.
- The remaining workspace must remain visible and scrollable.
- Every pop-up must have a clearly visible Cancel button at the top so the user can close it immediately.
- Avoid oversized panels and unnecessary screen obstruction.

The Edit toolbar above the keyboard is a special case: it must remain a thin, horizontally scrollable toolbar immediately above the keyboard.

41. Implementation and Testing Requirements

Test the main selected-text toolbar and Edit toolbar independently.

Verify that:

1. Tapping a text element displays its selection boundary.
2. The main toolbar appears and edits the entire text element.
3. Tapping Edit opens the separate keyboard-based editing environment.
4. The Edit toolbar appears immediately above the keyboard.

5. The Edit toolbar scrolls horizontally without obstructing the editing workspace.
6. Highlighted words can be replaced without replacing the rest of the text.
7. Individual words can be bolded, italicised, underlined and recoloured.
8. Whole-text-element formatting does not accidentally affect other pages.
9. Uploaded fonts remain available in the font library.
10. Light, Medium and Bold font weights visibly work.
11. Font size controls and custom size input work.
12. HEX colours and genuine multi-colour gradients render correctly.
13. Effects render correctly and respond to their adjustment controls.
14. Underlines have comfortable spacing.
15. Text Styles changes the selected element's recognised style.
16. Numbered and bulleted lists render and format correctly.
17. Letter spacing and line spacing respond to adjustments.
18. Page-specific edits remain confined to the current page unless an explicit multi-page operation is selected.

Do not implement controls as decorative placeholders. Every available control must perform its advertised function.

Preserve all previously confirmed application requirements and existing working functionality while implementing this section.

LITDOM BATCH STUDIO

Functional Specification — Version 4

Editor Tools: Text, Upload, Fields, Sync Layout and Spreadsheet Population

---

42. Text Tool: Adding Individual Design Elements

The **Text** tool allows users to add individual text boxes and other text elements to a design.

These additions are page-specific by default. Adding an element to one page must not automatically add it to other pages unless the user subsequently uses a syncing function.

The Text tool must contain the following options:

42.1. Add Text Box

- Adds a new, individually editable text box to the current page only.
- The user can enter and format its content.

- It must not automatically appear on other pages.

42.2. Add a Heading

- Adds a heading element to the current page.

42.3. Add Subheading

- Adds a subheading element to the current page.

42.4. Add Some Body Text

- Adds a body-text element to the current page.

42.5. Add Page Number

- Automatically generates page numbers across all pages of the design.
- Page numbers must correspond to the actual page sequence.

42.6. Add Table

- Adds a table to the current page.
- This option must remain available in the Text tool. It must not be accidentally omitted from the interface.

42.7. Add Horizontal Line

- Adds a horizontal line to the current page.

42.8. Font Combinations

Font Combinations provides coordinated typography presets, similar in purpose to the font-combination features in professional design applications.

Requirements:

- The initial system must include at least three professionally designed font-combination presets.
- Each combination should provide coordinated font choices for appropriate text roles, such as headings, subheadings and body text.
- Users must be able to create their own font combinations.
- Users must be able to export their custom combinations and import them again.
- Once imported, a combination must be saved in the application’s persistent font-combination library.
- Saved and imported combinations must remain available across future projects and editing sessions.
- Users must not need to re-import a combination every time they start a new design.

The font-combination library should be expandable without disrupting existing combinations.

---

43. Upload Tool

The **Upload** tool allows users to upload images from their device into the design.

Uploaded images can be added to the current project and used as design elements.

This function is separate from the initial background-image upload in the production workflows. It is an editor tool for adding images during the editing process.

---

44. Fields Tool

The **Fields** tool allows users to add separate text fields to a design.

It is particularly relevant to the Upload Images workflow, where the user may need additional spreadsheet-connected fields beyond the initially generated placeholders.

Examples include:

- Subtitle
- Heading 2
- Other supported text-field types

A field added through Fields becomes part of the design's field structure and can participate in spreadsheet matching and population.

Important distinction:

- Text tool: Adds individual text elements, such as a manually added text box, heading or body-text element.
- Fields tool: Adds a field to the design's structured field system.

These tools must not be treated as interchangeable.

---

45. Sync Layout

Sync Layout provides two separate operations. They must remain distinct because they perform different tasks.

45.1. Sync This Design's Layout

This operation synchronises the overall layout of a page with other pages.

The layout includes:

- Font choices and typography.
- Title, heading and other text styles.
- Font styling and presentation.
- Text positioning.
- Element arrangement.
- The overall organisation and visual structure of the page.

The user must be able to choose one of three scopes:

This Page

- Default option.
- Applies the layout to the current page only.

All

- Applies the selected page's overall layout across all pages.

Custom

- Allows the user to select specific pages.
- The layout is applied only to the selected pages.

45.2. Sync Text Field's Layout

This operation synchronises an individual text field across pages without synchronising the entire page layout.

Example: The user wants every Title field in the project to have the same styling and arrangement.

The process is:

1. Select the relevant text field.
2. Choose Sync Text Field's Layout.
3. Select the pages to synchronise with, such as all pages or specific pages.
4. Apply the synchronisation.

The system must:

- Identify the corresponding field on each selected page.

- Apply the selected field's styling and layout.
- Maintain the correct positioning and arrangement.
- Avoid modifying unrelated fields or other elements on those pages.

Important distinction:

Sync This Design's Layout affects the overall page design.

Sync Text Field's Layout affects only the selected field across the chosen pages.

Neither operation should automatically modify pages outside the selected scope.

---

46. Plus (+): Spreadsheet Upload, Matching and Population

The Plus (+) button allows users to upload a spreadsheet and initiate the matching and population process.

Its behaviour depends on the production workflow through which the user entered the editor.

46.1. Upload Document Workflow

The Document workflow has a special analysis and field-detection stage.

When the user initially uploads the DOCX and the system generates the spreadsheet, it performs document analysis and field detection.

The system displays:

"Analysis and fields detected successfully. Ready to populate."

The user can then press **Match and Populate**.

If the user later uploads a revised spreadsheet through the Plus button:

- The spreadsheet is uploaded.
- The system displays **"Spreadsheet uploaded successfully."**
- The user can press **Match and Populate**.
- The system uses the existing document analysis and matching information.

It must not unnecessarily repeat the initial document analysis.

46.2. Upload Images Workflow

When the user enters through Upload Images and uploads a spreadsheet using Plus:

- The system uploads the spreadsheet.
- It displays **“Spreadsheet uploaded successfully.”**
- The user presses **Match and Populate**.
- The system uses the established field structure and matching process to populate the design.

There is no separate document-analysis confirmation.

46.3. Upload Spreadsheet Workflow

When the user enters through Upload Spreadsheet and uploads a spreadsheet using Plus:

- The system uploads the spreadsheet.
- It displays **“Spreadsheet uploaded successfully.”**
- The user presses **Match and Populate**.
- The system uses the selected template’s existing field structure to populate the design.

There is no separate document-analysis confirmation.

46.4. Workflow-Specific Behaviour

The following distinction is mandatory:

Workflow	Upload confirmation	Separate analysis and field-detection confirmation	Next action
Upload Document	Spreadsheet uploaded successfully	Yes, during initial document processing	Match and Populate
Upload Images	Spreadsheet uploaded successfully	No	Match and Populate
Upload Spreadsheet	Spreadsheet uploaded successfully	No	Match and Populate

Only the Document workflow uses the additional analysis and field-detection stage.

The application must preserve this workflow-specific behaviour rather than forcing all three workflows through the same sequence.

Implementation Instructions

1. Preserve all previously specified editor tools and functionality.
2. Do not remove Add Table from the Text tool.
3. Keep individual text additions separate from structured field additions.
4. Ensure page-specific additions remain page-specific unless synchronisation is explicitly requested.
5. Implement both layout-synchronisation operations independently.
6. Ensure custom font combinations persist in the application’s library.
7. Implement spreadsheet upload and population according to the originating workflow.
8. Do not repeat document analysis when a revised spreadsheet is uploaded after the initial analysis has already been completed.
9. Test all three workflows independently to confirm that the correct upload messages and next actions appear.
10. Do not introduce additional controls or behaviour that have not been specified.

End of Version 4.

IMG_1495 (2).pdf

To improve the architecture, usability, and robustness of Litdom Batch Studio, consider the following strategic enhancements categorized by system layers:1. Robust Document Parsing & Error Resilience

- Web Workers for DOCX Parsing: Move the heavy OpenXML structure inspection and multi-page text extraction into a dedicated Web Worker. This prevents UI freezing when processing large multi-hundred-page manuscripts.
- Granular Extraction Reports: Instead of simply flagging extraction failures, provide an interactive "Review Exceptions" modal that highlights ambiguous custom styles or unrecognized formatting inline, allowing users to manually map them to one of the 27 standard style categories before final spreadsheet generation.

2. Enhanced State Management & Undo/Redo Precision

- Command Pattern for Undo/Redo: Implement a strict Command Pattern for the universal editor's history stack. Each mutation (text rewrite, font styling, position shift, resizing) should be encapsulated as an undoable command object containing both forward and reverse state diffs to guarantee flawless step-by-step restoration.
- Auto-Recovery Snapshots: Leverage IndexedDB to periodically save lightweight operational snapshots in the background, protecting users against unexpected browser crashes or mobile memory reclaims during heavy batch edits.

3. Advanced Typography & Rendering Fidelity

- Client-Side Gradient Engine: Implement an HTML5 Canvas / SVG shader pipeline for text rendering to ensure multi-color gradients blend smoothly across individual characters rather than bounding boxes.
- Font Fallback & Metric Normalization: When custom .ttf/.woff fonts are uploaded, automatically calculate baseline metrics and bounding boxes to prevent layout shifts or text overflow when swapping between custom weights (Light, Medium, Bold).

4. Optimized Mobile UX & Touch Targets

- Gesture-Based Canvas Controls: Enhance pinch-to-zoom and two-finger rotation on the mobile canvas, ensuring selection boundaries and resize handles scale dynamically based on viewport density.
- Contextual Keyboard Padding: Ensure the horizontally scrollable Edit toolbar above the mobile keyboard dynamically listens to viewport resize events (virtual keyboard appearance/dismissal) to eliminate layout thrashing.

Would you like me to help you draft the

SYSTEM DIRECTIVE: LITDOM BATCH STUDIO IMPLEMENTATION

You are an expert Full-Stack Web Application Architect and UI/UX Engineer. Your task is to build and maintain "Litdom Batch Studio" exactly as specified in the attached functional specifications PDF and visual assets.

CRITICAL CORE ARCHITECTURE & IMPLEMENTATION RULES:
1. **Local-First Architecture:** Use IndexedDB for all data persistence, project storage, asset handling, custom fonts, and storage usage calculations. Do not implement cloud databases, user logins, or server-side auth.
2. **The Three Independent Workflows:**
   - **Document Workflow:** DOCX upload -> 27-style structural scan -> Initial spreadsheet download -> Revised spreadsheet upload -> Validation -> Placeholder preparation -> User approval -> Population -> Universal Editor.
   - **Image Workflow:** Batch image uploads -> Auto-placeholders -> Layout design -> "Apply Layout to All" -> Spreadsheet upload -> Auto-population -> Universal Editor.
   - **Spreadsheet Workflow:** Spreadsheet upload -> Select existing application template or plain-white template -> Auto-population -> Universal Editor.
3. **Universal Dual-Level Text Editing:**
   - **Main Toolbar:** Modifies entire text elements as a whole (Fonts, Sizes, Colors, Gradients, Effects, Formats).
   - **Edit Toolbar (Pencil):** Opens a horizontally scrollable mobile keyboard toolbar for fine-grained, selection-dependent edits (formatting specific words, lowercase/uppercase/Mixed Formatting via long-press, individual word colors/bold/italic/underline).
4. **Layout Synchronization & Export Engine:**
   - Implement "Sync This Design’s Layout" (scopes: This Page, All, Custom) and "Sync Text Field’s Layout".
   - Export Engine must support PNG, JPG, WebP, and advanced PDF settings (Digital, RGB, true CMYK, Flatten, Crop Marks & Bleed) with page selectors (Current Page, All, Custom Select).

Build the complete, production-ready web application adhering strictly to every rule, data flow, and UI requirement outlined in the attached documentation.
-----Next Steps:

Would you like me to guide you on how to structure the project's folder layout or write the initial JavaScript code for the IndexedDB storage manager to get started in AI Studio?

SYSTEM ROLE & DIRECTIVE
You are an expert Full-Stack Web Application Architect and UI/UX Engineer. Your task is to build, maintain, and expand "Litdom Batch Studio"—a professional, mobile-first, local-first

document production and bulk design web application. You must implement the application with zero unnecessary external server dependencies, relying entirely on browser-based architecture (IndexedDB) and client-side processing.

---

1. CORE ARCHITECTURE & STORAGE (LOCAL-FIRST)
- **Database:** IndexedDB must be used for all project storage, asset persistence (background templates, images, custom fonts, binary blobs), and recovery snapshots.
- **State Management:** Fully reactive client-side state. Deleting a project must cleanly remove its associated binary files and metadata from IndexedDB and instantly update the dynamic storage usage meter.
- **No Cloud/Auth:** Do not implement user authentication, login screens, server-side databases, or subscription systems.

---

2. DASHBOARD & PROJECT MANAGEMENT
- **Landing Page Interface:** Implement the dashboard exactly as referenced in the visual specifications, featuring:
  - Header with branding ("LITDOM BATCH STUDIOS") and Search bar ("Search here").
  - "Create a Project" section with 3 independent entry cards:
    1. Upload DOCX (Document Workflow)
    2. Upload Images (Image Workflow)
    3. Upload Sheet (Spreadsheet Workflow)
  - Automatic File Count Detection: Do not force users to choose between "Single" or "Bulk" modes before uploading. The app must dynamically inspect file counts.
  - "Your Projects" Grid: Project preview cards featuring a top-left folder icon, top-right 3-bar menu, and preview thumbnail.
  - Project 3-Bar Menu Actions: Exactly 4 options: (1) Download, (2) Edit, (3) Delete, (4) Save as Template.
  - Storage Indicator: Real-time calculation of used vs. available storage capacity.

---

3. THE THREE INDEPENDENT WORKFLOWS

WORKFLOW A: Document Workflow (DOCX Pipeline)
1. **Upload & Parse:** Accept a `.docx` file. Perform a deterministic OpenXML structure inspection to extract paragraph styles using the 27 standard style categories (Title, Subtitle, Heading 1-9, Normal Text, Quotes, Body Text variants, Lists, Captions, etc.).
2. **Dynamic Spreadsheet Generation:**
   - Group consecutive identical styles into single occurrences.
   - Map document pages to spreadsheet rows (Row 1 = Headers, Row 2+ = Page 1, Page 2...).

- Dynamically generate shared columns based on the maximum occurrence count of each style across the entire document. Leave unused cells empty.
3. **Download & Revise:** Generate and download a real, editable Excel file.
4. **Template Upload:** User uploads background template images matching the page count.
5. **Revised Spreadsheet Upload:** User uploads the edited spreadsheet.
6. **Validation & Placeholder Preparation:** Validate structural changes (added/removed/renamed columns) and automatically prepare text placeholders on templates *before* content population.
7. **Approval & Population:** User reviews placeholders, clicks an explicit "Approve/Match and Populate" action, and enters the universal editor.

WORKFLOW B: Image Workflow (Batch Graphics)
1. Upload single or multiple background images of arbitrary dimensions.
2. System auto-places standard text placeholders on the first image.
3. User arranges layout and clicks **"Apply Layout to All"**.
4. User uploads a spreadsheet via the `+` control to populate data rows across the image batch.

WORKFLOW C: Spreadsheet Workflow (Template Generation)
1. User uploads a spreadsheet.
2. User selects an existing application template (or a saved user template, or a plain-white template).
3. System automatically validates column compatibility and populates designs.

---

4. UNIVERSAL EDITING INTERFACE & DUAL TEXT EDITING
All three workflows funnel into a single, mobile-optimized universal editor.

A. Element Selection & Main Toolbar
Tapping any text box displays a selection boundary (with resizing handles) and opens the main toolbar:
`[Edit] [Text Styles] [Font] [Font Size] [Color] [Effect] [Format]`
Rule: Actions here modify the **entire text element**.

B. Selected-Content Editing (The Edit Pencil)
Tapping the **Edit (Pencil)** button opens a specialized, horizontally scrollable toolbar directly above the mobile keyboard.
Rule: Actions here (formatting, color changes, bold, italic, underline, uppercase/lowercase, Mixed Formatting via long-press on Aa) apply **exclusively to highlighted text selections**, leaving unselected text untouched.
Page-Specific Rule: All edits are strictly page-specific by default. Edits on Page 1 never auto-propagate to Page 2 unless an explicit synchronization action is invoked.

C. Advanced Typography & Styling

- **Fonts:** Support custom font file uploads (.ttf/.woff) stored persistently in IndexedDB. Provide Light, Medium, and Bold weight options for every font.
- **Color & Gradients:** Support HEX inputs and genuine multi-color gradients (blending across text).
- **Effects:** Fully functional text effects (Drop Shadow, Glow, Hollow, Outline, Echo, Glitch) with adjustable parameters (intensity, offset, blur, colors).
- **Format:** Bulleted/numbered lists, letter spacing, line spacing, text alignment.

---

5. LAYOUT SYNCHRONIZATION & EXPORT SYSTEM

A. Sync Layout
- **Sync This Design's Layout:** Apply page structure across scopes: *This Page*, *All*, or *Custom* (specific pages).
- **Sync Text Field's Layout:** Propagate specific field styling (e.g., all Titles) across selected pages without altering other elements.

B. Export Engine
- **Image Exports (PNG, JPG, WebP):** Support custom resizing, quality settings (Low, Med, High), compression limits, and page selectors (*Current Page*, *All*, *Select* via thumbnail modal).
- **PDF Exports:** Support advanced settings: *Digital* (default), *RGB*, *CMYK* (true CMYK output verification), *Flatten*, and *Crop Marks & Bleed*.

---

6. CRITICAL IMPLEMENTATION CONSTRAINTS
1. Never silently discard data or use placeholder text for empty spreadsheet cells.
2. Maintain local-first IndexedDB architecture; do not require backend servers or user logins.
3. Ensure absolute fidelity to the 27 DOCX style categories without inventing unrequested styles.
4. Keep the three workflows modular and do not cross-contaminate their initialization steps.
-----What to do next:

Would you like me to guide you on how to structure your project files or write the specific initialization script for IndexedDB to get this started in AI Studio?

Fix the interface and canvas rendering issues across Litdom Batch Studio so that text, placeholders, and working elements appear crystal clear instead of blurry or washed out. Implement the following rendering corrections:

1. High-Density (Retina) Canvas Buffer Scaling:
   - Scale HTML5 canvas drawing buffers dynamically by window.devicePixelRatio.

- Set internal canvas width and height using the DPR multiplier while constraining CSS display size to match layout dimensions.
- Apply context scaling (ctx.scale(dpr, dpr)) to normalize coordinate systems and eliminate pixelation.

2. Hardware Acceleration & Sub-pixel Antialiasing:
- Ensure all CSS classes governing the workspace, canvas, and text elements explicitly include:
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-rendering: optimizeLegibility;
  transform: translateZ(0); (to force GPU layer promotion).

3. Integer Pixel Snapping:
- Update positioning, dragging, and resizing logic for text boxes and placeholders to round coordinates to whole integers (Math.round()) during rendering interactions, preventing sub-pixel interpolation blur.